



UNIVERSITAT POLITÈCNICA  
DE CATALUNYA  
BARCELONATECH

Manresa School of Engineering

---

# Real-Time Distributed ASL Fingerspelling Recognition System

Manresa, May 8, 2026

---

Master's Thesis (24 ECTS) submitted by

ERIC YAIR RAMIREZ CALDERON

in partial fulfillment of the requirements for the

Master's Degree in Machine Learning and Cybersecurity for IoT  
Systems (MERIT)

Advisors: Cristian Rodriguez Rivero  
Simona Ramanauskaitė  
Rajkumar Singh Rathore

Topics: Deep learning and computer vision; Edge-cloud distributed systems

This work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 License. To view a copy of this license, visit <https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode> or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

## **Declaration of Responsibility**

I, Eric Yair Ramirez Calderon, author of this Master's Thesis,

DECLARE

That this project and the accompanying report are original and solely the result of my work. Furthermore, all sources consulted have been included in the bibliography.

By submitting this report for evaluation, I consider this declaration signed for the purposes outlined in the agreement CG/2019/05/10, dated October 8, 2019, of the Governing Council of the Universitat Politècnica de Catalunya, which approves the procedure for plagiarism prevention.

Manresa, May 8, 2026



## Acknowledgments

I would like to express my sincere gratitude to my advisors, Cristian Rodriguez, Simona Ramanauskaitė, and Rajkumar Singh Rathore, for their guidance, feedback, patience, and encouragement throughout this project. Their support, insight, and trust were invaluable not only to the completion of this thesis, but also to my growth as a researcher and professional. I deeply appreciate the opportunity to learn from their experience and mentorship during this important stage of my academic journey.

The ASL Alphabet dataset used in this thesis was made publicly available by the Kaggle user *grassknotted* under a Creative Commons licence; I thank the original contributors for enabling reproducible research in this area.

I also thank the Universitat Politècnica de Catalunya for providing access to the computing infrastructure and network resources used during development and evaluation.

I would also like to thank the people whose work and personal example inspired the human-centred direction of this research, especially Beatriz and Renata, for reminding me that technology can and should be used to improve quality of life and social inclusion.

Finally, I am deeply grateful to my family and to the friends who became family for their constant support, encouragement, and presence throughout this journey. Although distance and life often kept us apart, their affection, trust, and support always remained close to me and gave me strength during the most challenging moments. This work is, in many ways, also theirs.



# Abstract

This thesis presents the design, implementation, and empirical evaluation of a real-time distributed ASL fingerspelling recognition system. The system distributes computational workload between a Raspberry Pi 5 edge device and a remote GPU server (NVIDIA RTX 4080 Super) connected via Wi-Fi and VPN. Three pipeline architectures: Baseline (MediaPipe + SVM), Option A (YOLOv8-pose + EfficientNet-B0), and Option B (end-to-end YOLOv8). They are compared on the 87,000-image ASL Alphabet dataset across model accuracy, end-to-end latency, network bandwidth, and edge-device resource utilisation. All three pipelines achieve high letter-level accuracy on the test set (Baseline 99.28 %, Option A 99.74 %, Option B 99.40 %), with mean end-to-end latencies of 65.6, 74.2, and 66.0 ms at a sustained  $\approx 11.3$  fps over the live distributed topology. Reducing JPEG quality from  $Q=95$  to  $Q=50$  cuts bandwidth by 71.3 % ( $5594 \rightarrow 1606$  kbps) at a  $-0.33$  pp accuracy cost; synthetic word accuracy with temporal smoothing reaches 98.7 % (Option A). Network transmission accounts for 46–51 % of total latency, identifying it (rather than model architecture) as the primary system-level bottleneck. These findings are bounded by the controlled dataset, a single Wi-Fi + VPN topology, and a non-expert signer in the live evaluation, which limits external validity to multi-signer and unconstrained-environment deployments. The contribution is empirical evidence that, on the controlled benchmark, accuracy ( $\geq 99$  %), real-time latency ( $\leq 66$  ms), and bandwidth efficiency (previously treated as mutually exclusive) can be simultaneously satisfied, and that the dominant lever for further improvement is network path quality rather than pipeline architecture.

**Keywords:** American Sign Language; fingerspelling; real-time gesture recognition; edge-cloud computing; Raspberry Pi 5; YOLOv8; EfficientNet-B0; distributed inference; network bandwidth; end-to-end latency.

# Resum

Aquesta tesi presenta el disseny, la implementació i l'avaluació empírica d'un sistema distribuït de reconeixement de dactilologia ASL en temps real. El sistema distribueix la càrrega computacional entre un dispositiu perifèric Raspberry Pi 5 i un servidor GPU remot (NVIDIA RTX 4080 Super) connectats mitjançant Wi-Fi i VPN. Es comparen tres arquitectures de pipeline: Baseline (MediaPipe + SVM), Opció A (YOLOv8-pose + EfficientNet-B0) i Opció B (YOLOv8 de punta a punta). Sobre el conjunt de dades ASL Alphabet de 87.000 imatges, en termes de precisió del model, latència de punta a punta, amplada de banda i utilització de recursos del dispositiu perifèric. Les tres pipelines assolixen una precisió alta a nivell de lletra al conjunt de prova (Baseline 99,28 %, Opció A 99,74 %, Opció B 99,40 %), amb latències mitjanes de

punta a punta de 65,6, 74,2 i 66,0 ms a un ritme sostingut de  $\approx 11,3$  fps sobre la topologia distribuïda real. Reduir la qualitat JPEG de  $Q=95$  a  $Q=50$  retalla l'amplada de banda en un 71,3 % ( $5594 \rightarrow 1606$  kbps) amb només  $-0,33$  pp de pèrdua de precisió; la precisió a nivell de paraula amb suavitzat temporal arriba al 98,7 % (Opció A). La transmissió per xarxa representa el 46–51 % de la latència total, fet que la identifica (per damunt de l'arquitectura del model) com el principal coll d'ampolla a nivell de sistema. Les conclusions estan acotades pel conjunt de dades controlat, una única topologia Wi-Fi + VPN i un signaire no expert en l'avaluació en viu, fet que limita la validesa externa cap a desplegaments multi-signaire o en entorns no controlats. La contribució principal és l'evidència empírica que, al banc de proves controlat, els objectius de precisió ( $\geq 99$  %), latència en temps real ( $\leq 66$  ms) i eficiència d'amplada de banda (tractats prèviament com a mútuament excloents) es poden satisfer simultàniament, i que la palanca dominant per a futures millores és la qualitat del camí de xarxa més que l'arquitectura del pipeline.

**Paraules clau:** Llengua de Signes Americana; dactilologia; reconeixement gestual en temps real; computació perifèrica-núvol; Raspberry Pi 5; YOLOv8; EfficientNet-B0; inferència distribuïda; ample de banda; latència extrem a extrem.



# List of Acronyms

|       |                                       |
|-------|---------------------------------------|
| AP    | Average Precision                     |
| API   | Application Programming Interface     |
| ARM   | Advanced RISC Machine                 |
| ASL   | American Sign Language                |
| BGR   | Blue-Green-Red (OpenCV channel order) |
| CER   | Character Error Rate                  |
| CI    | Confidence Interval                   |
| CNN   | Convolutional Neural Network          |
| COCO  | Common Objects in Context             |
| CPU   | Central Processing Unit               |
| CUDA  | Compute Unified Device Architecture   |
| CV    | Cross-Validation                      |
| FLOP  | Floating-Point Operation              |
| GPIO  | General-Purpose Input/Output          |
| GPU   | Graphics Processing Unit              |
| GRU   | Gated Recurrent Unit                  |
| GUI   | Graphical User Interface              |
| HTTP  | HyperText Transfer Protocol           |
| IOT   | Internet of Things                    |
| IP    | Internet Protocol                     |
| JPEG  | Joint Photographic Experts Group      |
| LR    | Learning Rate                         |
| LSTM  | Long Short-Term Memory                |
| MCP   | Metacarpophalangeal joint             |
| MJPEG | Motion JPEG                           |
| MTU   | Maximum Transmission Unit             |
| OS    | Operating System                      |
| PC    | Personal Computer                     |
| PIP   | Proximal Interphalangeal joint        |
| POST  | HTTP POST (data submission method)    |
| RAM   | Random Access Memory                  |
| RBF   | Radial Basis Function                 |
| REST  | Representational State Transfer       |
| RGB   | Red-Green-Blue                        |
| RNN   | Recurrent Neural Network              |
| SD    | Standard Deviation                    |
| SGD   | Stochastic Gradient Descent           |
| SLR   | Sign Language Recognition             |
| SRQ   | Sub-Research Question                 |
| SVM   | Support Vector Machine                |
| VGG   | Visual Geometry Group                 |
| VPN   | Virtual Private Network               |
| VRAM  | Video Random Access Memory            |
| WER   | Word Error Rate                       |
| YOLO  | You Only Look Once                    |

# Contents

|                                                                                       |             |
|---------------------------------------------------------------------------------------|-------------|
| <b>Abstract</b>                                                                       | <b>v</b>    |
| <b>Resum</b>                                                                          | <b>v</b>    |
| <b>List of Acronyms</b>                                                               | <b>viii</b> |
| <b>1. Introduction</b>                                                                | <b>1</b>    |
| 1.1. Problem Statement . . . . .                                                      | 1           |
| 1.2. Research Gap . . . . .                                                           | 2           |
| 1.3. Research Questions . . . . .                                                     | 3           |
| 1.4. Contributions . . . . .                                                          | 4           |
| <b>2. Related Work</b>                                                                | <b>5</b>    |
| 2.1. Background . . . . .                                                             | 5           |
| 2.1.1. ASL Fingerspelling as a Recognition Task . . . . .                             | 5           |
| 2.1.2. Core Deep Learning Architectures . . . . .                                     | 6           |
| 2.1.3. Edge Computing with Raspberry Pi . . . . .                                     | 6           |
| 2.2. Literature Review . . . . .                                                      | 7           |
| 2.2.1. State of the Art in ASL and Sign Language Fingerspelling Recognition . . . . . | 8           |
| 2.2.2. Deep Learning Architectures for Resource-Constrained Deployment . . . . .      | 9           |
| 2.2.3. Distributed Edge-Cloud Architectures and Task Offloading . . . . .             | 10          |
| <b>3. Methodology</b>                                                                 | <b>11</b>   |
| 3.1. Theoretical Framework . . . . .                                                  | 11          |
| 3.1.1. Key Constructs and Definitions . . . . .                                       | 11          |
| 3.1.2. Hypothesised Relationships . . . . .                                           | 12          |
| 3.1.3. Modelling Perspective . . . . .                                                | 13          |
| 3.2. Research and Experimental Design . . . . .                                       | 13          |
| 3.2.1. The Three Pipeline Configurations . . . . .                                    | 13          |
| 3.2.2. Network Architecture and the Edge-Cloud Split . . . . .                        | 14          |
| 3.2.3. Word Formation Evaluation . . . . .                                            | 15          |
| 3.2.4. Factors, Conditions, and Controls . . . . .                                    | 15          |
| 3.3. Data Collection and Preprocessing . . . . .                                      | 16          |
| 3.3.1. Dataset . . . . .                                                              | 16          |
| 3.3.2. Preprocessing Pipeline . . . . .                                               | 17          |
| 3.4. Methods and Analysis Procedure . . . . .                                         | 17          |
| 3.4.1. Baseline: MediaPipe + SVM . . . . .                                            | 17          |
| 3.4.2. Option A: YOLOv8-pose + EfficientNet-B0 . . . . .                              | 17          |
| 3.4.3. Option B: Custom YOLOv8 End-to-End . . . . .                                   | 18          |
| 3.4.4. Buffer-Based Word Formation (Baseline) . . . . .                               | 18          |
| 3.4.5. Training Infrastructure . . . . .                                              | 18          |

|           |                                                                     |           |
|-----------|---------------------------------------------------------------------|-----------|
| 3.5.      | Evaluation Procedure and Metrics . . . . .                          | 18        |
| 3.5.1.    | Model-Level Metrics . . . . .                                       | 19        |
| 3.5.2.    | System-Level Metrics . . . . .                                      | 20        |
| 3.5.3.    | Secondary Experiment: Bandwidth–Accuracy Trade-off (SRQ2) . . . . . | 21        |
| 3.6.      | Reliability and Validity . . . . .                                  | 21        |
| 3.6.1.    | Reliability . . . . .                                               | 21        |
| 3.6.2.    | Threats to Validity . . . . .                                       | 22        |
| <b>4.</b> | <b>Experimental Setup</b>                                           | <b>23</b> |
| 4.1.      | Dataset Description and Exploratory Analysis . . . . .              | 23        |
| 4.1.1.    | Class Balance . . . . .                                             | 23        |
| 4.1.2.    | MediaPipe Detection Recall by Class . . . . .                       | 23        |
| 4.1.3.    | Representative Samples . . . . .                                    | 24        |
| 4.2.      | Implementation Environment . . . . .                                | 24        |
| 4.2.1.    | Hardware . . . . .                                                  | 24        |
| 4.2.2.    | Software Versions . . . . .                                         | 25        |
| 4.3.      | Hyperparameter Configuration . . . . .                              | 25        |
| 4.3.1.    | Baseline: SVM . . . . .                                             | 25        |
| 4.3.2.    | Option A: YOLOv8n-pose (Hand Detection Stage) . . . . .             | 25        |
| 4.3.3.    | Option A: EfficientNet-B0 (Classification Stage) . . . . .          | 27        |
| 4.3.4.    | Option B: YOLOv8n Detect (End-to-End) . . . . .                     | 27        |
| 4.4.      | Live Evaluation Protocol . . . . .                                  | 27        |
| 4.4.1.    | Network Topology . . . . .                                          | 28        |
| 4.4.2.    | End-to-End Latency Benchmark . . . . .                              | 28        |
| 4.4.3.    | Word Accuracy Live Evaluation . . . . .                             | 28        |
| 4.5.      | Threats to Validity . . . . .                                       | 28        |
| 4.5.1.    | Construct Validity . . . . .                                        | 28        |
| 4.5.2.    | Internal Validity . . . . .                                         | 29        |
| 4.5.3.    | External Validity . . . . .                                         | 29        |
| 4.5.4.    | Statistical Conclusion Validity . . . . .                           | 29        |
| 4.5.5.    | Residual Risks . . . . .                                            | 30        |
| <b>5.</b> | <b>Results</b>                                                      | <b>31</b> |
| 5.1.      | Offline Letter-Level Accuracy . . . . .                             | 31        |
| 5.2.      | End-to-End Latency . . . . .                                        | 31        |
| 5.3.      | Edge-Device Resource Utilisation . . . . .                          | 35        |
| 5.4.      | Word Accuracy . . . . .                                             | 35        |
| 5.5.      | Bandwidth–Accuracy Trade-off . . . . .                              | 38        |
| 5.6.      | Ablation Analysis: Component Contributions . . . . .                | 39        |
| 5.7.      | Hypothesis Evaluation Summary . . . . .                             | 40        |
| <b>6.</b> | <b>Discussion</b>                                                   | <b>41</b> |
| 6.1.      | Comparison with Prior Work . . . . .                                | 41        |
| 6.1.1.    | Model-Level Accuracy . . . . .                                      | 41        |
| 6.1.2.    | System-Level Latency . . . . .                                      | 42        |
| 6.1.3.    | Sequence-Level Evaluation . . . . .                                 | 42        |

|           |                                                                               |           |
|-----------|-------------------------------------------------------------------------------|-----------|
| 6.2.      | Interpretation of Key Findings . . . . .                                      | 42        |
| 6.2.1.    | All Three Pipelines Saturate the Dataset . . . . .                            | 42        |
| 6.2.2.    | Detection Recall is the Accuracy Driver, Not Classifier Capacity . . . . .    | 43        |
| 6.2.3.    | Network Latency Dominates: Architecture Choices Have Bounded Impact . . . . . | 43        |
| 6.2.4.    | Temporal Smoothing is as Important as Classifier Accuracy . . . . .           | 43        |
| 6.2.5.    | JPEG Quality Reduction is Bandwidth-Efficient Without Accuracy Cost . . . . . | 43        |
| 6.3.      | Reflection on Design Decisions . . . . .                                      | 44        |
| 6.4.      | Validity, Reliability, and Generalisability . . . . .                         | 44        |
| 6.5.      | Limitations . . . . .                                                         | 45        |
| 6.6.      | Theoretical and Practical Implications . . . . .                              | 46        |
| 6.7.      | Summary . . . . .                                                             | 46        |
| <b>7.</b> | <b>Conclusions and Future Work</b>                                            | <b>49</b> |
| 7.1.      | Recap of the Problem and Scientific Relevance . . . . .                       | 49        |
| 7.2.      | Answers to the Research Questions . . . . .                                   | 49        |
| 7.3.      | How Limitations Qualify the Conclusions . . . . .                             | 51        |
| 7.4.      | Contribution Relative to the State of the Art . . . . .                       | 52        |
| 7.5.      | Future Work . . . . .                                                         | 52        |
|           | <b>Bibliography</b>                                                           | <b>55</b> |
| <b>A.</b> | <b>Hyperparameter Grids and Training Configuration</b>                        | <b>57</b> |
| A.1.      | Baseline: SVM Grid . . . . .                                                  | 57        |
| A.2.      | Option A: EfficientNet-B0 Training Configuration . . . . .                    | 57        |
| A.3.      | Option B: YOLOv8n-detect Training Configuration . . . . .                     | 57        |
| A.4.      | Data Augmentation and Preprocessing Summary . . . . .                         | 58        |



# 1. Introduction

American Sign Language (ASL) serves as a primary communication method for a large community worldwide, and fingerspelling is one of its most critical components, it is how signers convey proper names, technical terms, and concepts that do not have an established sign. Recognition technology for ASL has obvious potential to bridge communication gaps, but building systems that actually work in real time has proven harder than the research community initially expected. The core difficulties are computational: the models that recognise signs well tend to be too heavy for portable hardware, and the hardware that is portable tends to be too weak for accurate models.

This tension between accuracy, latency, and portability is well known, yet the literature has largely addressed each dimension in isolation. The present work investigates whether a distributed edge-cloud architecture can resolve the tension by splitting computational stages between an affordable Raspberry Pi 5 and a remote GPU server. The scientific contribution is not the individual components (each has been demonstrated elsewhere) but the systematic, multi-objective comparison of three pipeline architectures under identical distributed conditions, producing quantitative evidence on which combinations of accuracy, latency, and resource utilisation are simultaneously achievable and which remain mutually exclusive.

## 1.1. Problem Statement

Current ASL recognition systems face what can be described as an accessibility paradox. The most accurate recognition models need powerful GPUs, which means expensive, non-portable, power-hungry machines. The people who stand to benefit most from this technology (those who rely on sign language daily) are the least likely to have access to that kind of hardware. Portable alternatives exist, but they tend to sacrifice too much accuracy or respond too slowly to be genuinely useful.

There is a more specific technical tension as well. Processing everything on an edge device (say, a Raspberry Pi) keeps the system portable, but the computational budget is extremely tight, too tight for the deep learning architectures that achieve state-of-the-art accuracy. Offloading everything to a cloud server solves the compute problem but introduces network latency, and if the round-trip delay exceeds a few hundred milliseconds the system stops feeling real-time.

This thesis investigates whether a distributed approach (splitting work between an affordable edge device and a remote GPU server) can achieve what single-dimension approaches cannot: high recognition accuracy, low end-to-end latency, and practical portability all at once. The specific question is whether strategically distributing computational stages, while systematically comparing multiple deep learning architectures, yields measurably better results than the approaches in the existing literature that optimise for one objective at a time.

To anchor this investigation in concrete terms, we define success along three measurable dimensions. For recognition accuracy, the target is letter-level performance within 3 percentage

points of the offline ceiling established by Alsharif et al. [Als+23] (99,95% for EfficientNet on the same dataset). For latency, the system must sustain  $\geq 15$  fps ( $\leq 66$  ms per frame), the real-time threshold operationally confirmed by Panopoulos et al. [Pan+25]. For portability, all edge-device operations must run within the thermal and memory envelope of a Raspberry Pi 5 without hardware modifications. A system that meets all three thresholds simultaneously (rather than trading one for another) would represent a qualitative advance over the single-dimension approaches reviewed in Section 1.2.

## 1.2. Research Gap

The existing literature on ASL recognition has, for the most part, tackled three problem areas separately:

1. Deep learning models for improved recognition accuracy.
2. Mobile and embedded implementations for portability.
3. Cloud-based processing for computational power.

**Deep learning models for improved recognition accuracy.** Recent work using CNNs like MobileNetV2 and EfficientNet-Lite has pushed letter recognition accuracy on ASL datasets to high levels, EfficientNet reaches 99,95% [Als+23] and SignNet-Nano reaches 98,5% on Raspberry Pi 4 [Zab+], when tested on mobile devices. However, these lightweight models sacrifice accuracy for computational efficiency, and even then, latency on ARM processors can limit real-time performance. What is striking when reviewing the literature is how many architectures remain untested for this specific task. YOLOv8-based pose estimation, for example, has shown strong results in general hand pose detection [Rey+; Pan+25] but has not been applied to sign language recognition. End-to-end YOLO models trained directly for ASL letter detection have not been compared side-by-side against the more traditional two-stage pipeline of detection followed by classification. Temporal sequence processing with LSTMs or GRUs [But+23; Raz+23] (which could help smooth letter transitions and reduce false detections) has received surprisingly little attention in distributed deployment scenarios. The question of which architecture best balances model complexity, accuracy, and computational cost for real-time ASL fingerspelling under resource constraints is still open.

**Mobile and embedded implementations for portability.** Several studies have deployed ASL recognition on resource-constrained IoT devices and shown it is technically possible [Zab+; Beg+23; Pra+24]. But the performance limitations are significant: high inference latency, near-100% CPU utilisation, and model capacity so restricted that more sophisticated architectures or any kind of language processing simply cannot run. What has not been adequately characterised is the boundary, which tasks can realistically run on a Raspberry Pi, and which need to be offloaded.

**Cloud-based processing for computational power.** Cloud-based approaches use powerful GPU infrastructure to run sophisticated models and typically achieve higher per-letter accuracy [Had+18; Ama+23]. The problem is deployment. Network latency adds up [Mag+19],

transmission time plus server processing time can easily exceed the threshold where the system feels responsive. These approaches also require a continuous, reasonably fast internet connection, which creates accessibility barriers of its own.

What nobody has done, as far as we could find, is combine all three dimensions in a single study: systematically evaluate multiple deep learning architectures for their accuracy-latency trade-offs, deploy them on affordable IoT edge hardware with quantified resource constraints, and optimise the task distribution between edge and cloud to achieve high accuracy and low latency simultaneously. That is the gap this thesis targets.

Stated more precisely, the gap spans three complementary levels. At the *empirical level*, no study reports both model-level classification metrics (accuracy, precision, recall, F1) and system-level distributed metrics (end-to-end latency, bandwidth, edge resource utilisation) for ASL fingerspelling under identical conditions, closing this gap requires a controlled experiment that measures all nine dependent variables simultaneously (Chapter 3). At the *methodological level*, the three pipeline families examined here (landmark-based, two-stage detection-classification, and end-to-end detection) have never been compared on a common dataset, common hardware, and a common evaluation protocol (closing this gap requires a fair, multi-seed comparison with uncertainty estimates. At the *system level*, no prior work characterises the Raspberry Pi 5 as an edge node for ASL recognition with quantified CPU, RAM, and bandwidth overhead) closing this gap requires live instrumentation of the full distributed pipeline. The research questions in Section 1.3 map directly onto these three levels, and the evaluation framework in Chapter 3 is designed so that each gap is addressed by at least one dependent variable with a literature-grounded comparison point.

### 1.3. Research Questions

The primary research question is:

*How does an integrated approach combining deep learning model optimisation, IoT edge device implementation, and distributed edge-cloud architecture compare to single-dimension approaches in achieving high accuracy, low latency, and practical portability for real-time ASL fingerspelling recognition?*

This breaks down into four sub-questions, each with concrete measurable aspects:

- SRQ1 — **Task Distribution and IoT Device Capabilities.** How should computational tasks be distributed between IoT edge devices and cloud infrastructure to balance recognition accuracy, system latency, bandwidth requirements, and edge device resource constraints?
- SRQ2 — **Pre-Processing Optimisation for Bandwidth Reduction.** What pre-processing methods can reduce data transmission volume while maintaining sufficient recognition accuracy for reliable ASL fingerspelling detection across different deep learning architectures?
- SRQ3 — **Deep Learning Architecture Selection and Sequence Processing.** What combination of hand detection, classification, and temporal sequence processing methods achieves the best balanced performance (accuracy, latency, resource utilisation) for distributed ASL fingerspelling recognition?

SRQ4 — Integrated System Performance Evaluation. How does the complete integrated system (best-performing deep learning architecture plus IOT edge implementation plus distributed task allocation) perform compared to baseline approaches focusing on single dimensions across key technical metrics?

To answer the research questions outlined above, it is necessary to first establish the theoretical and empirical foundations that inform the design of the proposed system. Each research question targets a specific dimension (model architecture, edge deployment, and distributed processing) which has been studied separately in prior work. Chapter 2 therefore reviews the relevant background concepts and existing literature across these areas, providing the basis for identifying limitations in current approaches and justifying the methodological choices made in this thesis.

## 1.4. Contributions

This thesis makes the following contributions:

- **First controlled multi-architecture comparison for distributed asl finger-spelling.** Three complete pipeline architectures (Baseline: MediaPipe + SVM; Option A: YOLOv8-pose + EfficientNet-B0; Option B: end-to-end YOLOv8) are evaluated on the same dataset, the same hardware, and the same live network topology, removing the cross-study confounds that prevent direct comparison of published results.
- **Empirical evidence that accuracy, latency, and bandwidth efficiency are simultaneously achievable.** All three pipelines meet a  $\geq 99\%$  letter-level accuracy target and a  $\leq 66$  ms end-to-end latency target at  $\approx 11.3$  fps over a live Pi-to-server topology, showing that the trade-offs assumed in prior single-dimension studies are not fundamental constraints of the distributed architecture.
- **Bandwidth–accuracy Pareto characterisation.** A systematic experiment varying JPEG quality ( $Q \in \{95, 75, 50, 25\}$ ) and frame rate quantifies the bandwidth–accuracy frontier for the first time in an ASL distributed system: reducing  $Q$  from 95 to 50 cuts bandwidth by 71.3 % ( $5594 \rightarrow 1606$  kbps) at a cost of only  $-0.33$  pp in letter accuracy.
- **Detection recall as the dominant accuracy driver.** Ablation analysis across the three pipelines reveals that class-level accuracy differences are explained by hand-detection recall rather than classifier capacity, a diagnostic finding that guides where engineering effort should be directed in future deployments.

## 2. Related Work

This chapter lays out the theoretical and empirical foundations for the thesis. It is split into two parts. Section 2.1 covers the background knowledge a reader needs to follow the methods and evaluation choices in later chapters: the characteristics of ASL fingerspelling as a recognition task, the deep learning architectures used as building blocks, and the edge-computing platform that defines hardware constraints. Section 2.2 reviews the prior studies most directly relevant to the three technical dimensions of this work: deep learning architecture optimisation (SRQ3), IOT edge deployment (SRQ1, SRQ2), and distributed edge-cloud task allocation (SRQ1, SRQ2, SRQ4). Each subsection in the literature review opens with a brief orientation, surveys the key studies (comparing their techniques, datasets, evaluation protocols, and reported results) and closes with a synthesis that identifies the specific gap this thesis addresses. The state-of-the-art benchmarks established in this chapter (Alsharif et al. [Als+23] for offline model-level accuracy; Panopoulos et al. [Pan+25] for system-level distributed latency; Saputra et al. [Sap+24] for sequence-level evaluation metrics) serve as the reference points against which the results of this thesis will be interpreted.

### 2.1. Background

Three prerequisite areas need to be established before the methodology can be properly understood: the characteristics of ASL fingerspelling as a recognition task, the deep learning architectures that serve as building blocks, and the edge-computing platform that defines the hardware constraints. Each area directly influences both the design choices in Chapter 3 and the way experimental results should be interpreted: the task definition determines which metrics are appropriate, the architectural properties determine the expected accuracy-latency trade-offs, and the hardware constraints determine which pipeline stages can run on the edge device versus the server.

#### 2.1.1. ASL Fingerspelling as a Recognition Task

Sign language recognition (SLR) covers two fundamentally different sub-problems. *Static gesture recognition* deals with fixed hand shapes captured in a single frame. *Dynamic gesture recognition* involves motion trajectories and sequential patterns across multiple frames [Pan+25; But+23]. ASL fingerspelling falls into the static category: each of the 26 letters of the American manual alphabet corresponds to a distinct hand configuration that is largely static, which means letter-level accuracy and per-frame inference speed are the performance criteria that matter most.

This distinction has direct consequences for system architecture. Static recognition can achieve high accuracy with single-frame CNN classifiers or detection-then-classification pipelines, while dynamic recognition requires temporal modelling through LSTM or GRU networks [But+23; V+26]. This thesis focuses squarely on the static fingerspelling task. LSTM-

based temporal sequence processing is considered an optional extension, useful for smoothing letter-boundary ambiguities, but not a core requirement of the system.

The static modelling assumption has a direct consequence for evaluation. Two letters of the ASL alphabet (J and Z) involve motion trajectories that a single frame cannot fully capture (see Figure 4.2 for representative hand shapes across all 26 letters). Any static classifier will see only a snapshot of these gestures, which means that reported accuracy on J and Z may overestimate performance on naturally signed versions and underestimate the difficulty a deployed system would face. We therefore report per-class results for J and Z separately (§3.5.1) so that readers can assess whether the static assumption introduces a systematic bias on these two classes.

### 2.1.2. Core Deep Learning Architectures

Three architectural families matter for this work:

*YOLO-based detectors* (YOLOv5, YOLOv8, YOLO11) are single-stage object detectors. They skip the region-proposal step entirely, which gives them the frame rates needed for real-time edge deployment [Pan+25; Beg+23; Zha+23] — Buttar et al. [But+23] report 28,9 fps at 96% mAP for static ASL signs using YOLOv6. YOLOv8 (the Ultralytics PyTorch-native iteration) introduces anchor-free detection and C2f modules that improve the accuracy-efficiency balance compared to earlier versions [Pan+25]. We selected YOLOv8 specifically because it is PyTorch-native, which simplifies integration with the rest of the pipeline.

*Lightweight CNN classifiers* (MobileNet, EfficientNet, SqueezeNet) use depthwise separable convolutions and inverted residual blocks to cut parameter counts by roughly 8–10× compared to full-scale ResNet or VGG backbones, while still maintaining good accuracy [Sap+24; Ama+23]. EfficientNet-B0 stands out for having a particularly favourable accuracy-per-FLOP ratio. Alsharif et al. [Als+23] report 99,95% top-1 accuracy for EfficientNet on the ASL Alphabet dataset, which establishes the model-level performance ceiling for this architecture on the dataset used in this thesis.

*Temporal sequence models* (LSTM, GRU) capture frame-to-frame dependencies, which is important for smoothing classification outputs and resolving coarticulation ambiguities in continuous fingerspelling [But+23; Lu+23]. Standard LSTMs address the vanishing gradient problem of vanilla RNNs; bidirectional variants can additionally capture future context [Raz+23]. However, Buttar et al. [But+23] found that their LSTM component achieved only 52% recall for dynamic signs, highlighting the difficulty of this approach in practice. In this thesis, LSTM integration is evaluated as an optional extension layered on top of the buffer-based Sequence-Processor already present in the baseline system.

### 2.1.3. Edge Computing with Raspberry Pi

Edge devices for SLR prototypes span a wide spectrum. At one extreme, laptops and desktop PCs offer ample compute but eliminate the resource-constrained scenario the problem statement targets: any pipeline would perform acceptably, and the accessibility paradox identified in Section 1.1 would never surface. At the other extreme, microcontrollers such as Arduino lack the memory and compute budget required to run even a lightweight CNN inference pipeline. Smartphones sit between these extremes but introduce additional complexity: closed operating environments (Android/iOS) and managed runtimes add overhead, camera APIs differ across manufacturers, and battery constraints create extra experimental variables. Single-board computers occupy a practical middle ground: they run a full Linux stack, expose GPIO interfaces

for direct camera integration, use ARM Cortex-A processors representative of a wide class of deployed IoT edge nodes, and cost under \$80, keeping the system accessible to the communities that benefit most.

Raspberry Pi single-board computers have become something of a standard platform for academic SLR prototypes. The reasons are straightforward: low cost, GPIO interfaces for hardware integration, and ARM Cortex-A processors that are representative of real edge deployment conditions [Pan+25; Zab+; Pra+24]. For this thesis we use the Raspberry Pi 5 (ARM Cortex-A76, 8 GB RAM), which offers roughly 2–3× the CPU throughput of the Pi 4 while keeping the same form factor and price point.

Pi hardware imposes well-documented constraints. CPU throughput is approximately 1/100th of a modern discrete GPU. There is no dedicated neural processing unit in the standard configuration. Memory bandwidth is limited, and thermal throttling under sustained inference loads is a known issue [Zaf+25; Als+23; Che+]. These limitations motivated three of the key design decisions in this thesis: choosing lightweight models, offloading compute-heavy operations to the GPU server, and explicitly measuring CPU utilisation and thermal behaviour as experimental variables.

In practical terms, these constraints mean that the Pi 5 is expected to become the bottleneck in two specific scenarios: (a) if JPEG encoding at high quality levels saturates the CPU, edge-side latency ( $T_{\text{edge}}$ ) will dominate end-to-end latency regardless of how fast the server processes inference; and (b) if sustained frame capture drives RAM consumption above the available 8 GB, the system will become unstable under prolonged operation. The system-level evaluation in §3.5.2 is designed to detect both failure modes through continuous CPU and RAM monitoring at 1 Hz over 60 s steady-state windows, with results reported as mean and peak values alongside 95th percentile latency to characterise tail behaviour.

Taken together, these foundational elements explain why the methodology settles on YOLOv8 as the detection backbone, EfficientNet-B0 as the classification head, and a split architecture where the Pi 5 handles capture and transmission while the RTX 4080 Super workstation does the heavy inference.

The three families occupy different positions in the accuracy-latency design space, and these positions shape the hypotheses tested in Chapter 3. YOLO-based detectors offer the fastest per-frame inference (Panopoulos et al. [Pan+25] report 36 ms to 38 ms on a GPU server) but sacrifice some classification granularity because the detection head must simultaneously localise and classify. Lightweight CNN classifiers like EfficientNet-B0 achieve the highest reported per-class accuracy on the ASL Alphabet dataset (99.95%, Alsharif et al. [Als+23]) but require a separate detection stage, adding latency. Temporal models add robustness to frame-level noise at the cost of an additional processing step and an assumption of sequential input that the static dataset cannot fully satisfy. Understanding these trade-offs is essential for interpreting why Option A is expected to achieve the highest accuracy, Option B the lowest latency, and the Baseline the most predictable edge-device behaviour (H1–H3 in §3.1.2).

## 2.2. Literature Review

The three subsections below review prior work organised by theme, each aligned with specific research questions. Each subsection opens with a brief orientation, surveys the key studies, and closes with a synthesis that identifies the specific gap this thesis addresses. Beyond summarising individual contributions, the review explicitly compares how different studies approach the

same problem (which datasets they use, which evaluation protocols they follow, what metrics they report, and what performance levels they achieve) so that the reader can assess whether published results are directly comparable and where methodological differences prevent fair comparison. This cross-study perspective is essential for positioning the methods chosen in this thesis and for establishing the expected performance ranges that will guide the interpretation of results in later chapters.

### 2.2.1. State of the Art in asl and Sign Language Fingerspelling Recognition

*Relevant to: SRQ3 (architecture selection) and SRQ4 (integrated performance evaluation).*

The goal here is to establish the performance baselines against which this thesis will be evaluated.

Several recent systems have demonstrated high-accuracy static recognition. Panopoulos et al. [Pan+25] achieved 99,07% mAP50 at 23,4 fps for Greek Sign Language using YOLO11X-seg deployed on a Pi-to-GPU-server architecture. Their system provides the most directly comparable system-level benchmark for this thesis, same Pi-to-GPU-server split, same JPEG-based frame transmission, and they provide an explicit latency decomposition (inference 36 ms to 38 ms + preprocessing 1,6 ms + postprocessing  $\sim 3$  ms = 42,7 ms total). Zabin et al. [Zab+] reported 98,5% accuracy at 15 fps with SignNet-Nano on Raspberry Pi 4, which shows that edge-only deployment is at least feasible for static gestures. Razman et al. [Raz+23] found that YOLOv5 achieved 97% accuracy with the lowest training time (10 minutes) when compared against LSTM and CNN-based classifiers.

Alsharif et al. [Als+23] provide the model-level classification benchmark that matters most for this thesis. They used the identical 87,000-image ASL Alphabet dataset from Kaggle and evaluated five architectures (AlexNet, ConvNeXt, EfficientNet, ResNet-50, and Vision Transformer) reporting accuracy, precision, recall, and F1-score for each. EfficientNet reached 99,95% accuracy, 99,90% precision, 99,94% recall, and 99,92% F1; ResNet-50 reached 99,98%. These results set the offline classification ceiling for the dataset used here. Their metric definitions (Equations 2–5 in their paper) provide the formal basis for the model-level evaluation protocol in §3.5.1. Notably, Alsharif et al. operated entirely offline on a single machine and explicitly listed converting their models into a real-time system as future work [Als+23], which is essentially the gap this thesis sets out to fill.

When it comes to dynamic signing and sentence-level recognition, performance drops significantly. Saputra et al. [Sap+24] achieved 49,29% sentence accuracy with 18,8 ms inference time for Indonesian Sign Language (Bisindo) using a YOLOv5 + MobileNetV2 + LSTM pipeline. Beyond speed and accuracy, they separately report a word error rate (WER) of 16,42% and a SacreBLEU score of 67,77%. This demonstrates that sequence-level outcome metrics (measuring whether a complete unit of communication was produced correctly) provide information that frame-level accuracy alone cannot capture. This two-layer approach (binary outcome plus degree-of-error) is what motivated the decision to use both Word Accuracy and Character Error Rate for the word formation evaluation in §3.2.3.

Buttar et al. [But+23] built a hybrid YOLOv6+LSTM system achieving 96% mAP at 28,9 fps for static signs and 80% precision for dynamic signs, confirming that unified architectures are technically possible but involve careful trade-offs. Debnath et al. [Deb+24] used MediaPipe Holistic landmarks as input to a fine-tuned LSTM and achieved real-time performance. Their architecture closely mirrors the MediaPipe + SVM Baseline that serves as our starting point.

**Synthesis.** The classification benchmark for ASL fingerspelling is well-established at the model level, Alsharif et al. [Als+23] report EfficientNet at 99,95% accuracy on the 87,000-image ASL Alphabet dataset under offline conditions. At the system level, Panopoulos et al. [Pan+25] demonstrate a Pi-to-GPU-server architecture achieving 42,7 ms processing at 23,4 fps. But no study bridges these two baselines. We could not find any published work that reports both model-level classification metrics (accuracy, precision, recall, F1) and system-level distributed metrics (end-to-end latency, bandwidth, edge resource utilisation) in a single controlled experiment comparing multiple pipeline architectures for ASL fingerspelling. This is the gap that motivates SRQ3 and SRQ4.

A cross-study comparison reveals an important methodological limitation: evaluation protocols differ substantially across the works reviewed above, which makes direct comparison unreliable. Alsharif et al. [Als+23] use five-fold cross-validation with the Adamax optimiser on  $200 \times 200$  images and report accuracy, precision, recall, and F1. Panopoulos et al. [Pan+25] use mAP50 and mAP50-95 on a different dataset (Greek Sign Language, 1,669 images) with a different model family (YOLO11X-seg). Saputra et al. [Sap+24] report sentence accuracy and WER on Indonesian Sign Language video sequences, a dynamic task fundamentally different from static fingerspelling. These differences mean that published performance numbers cannot simply be placed side by side; they must be interpreted within the context of each study’s dataset, task definition, and hardware. This thesis addresses this comparability problem by evaluating all three pipeline architectures on the *same* dataset, *same* hardware, *same* split, and *same* metric set, the controlled comparison that the literature currently lacks.

### 2.2.2. Deep Learning Architectures for Resource-Constrained Deployment

*Relevant to: SRQ3 (architecture selection and sequence processing).*

This subsection looks at how researchers have optimised deep learning models for execution on devices with constrained compute, memory, and power budgets. The findings here directly informed our model selection decisions.

YOLO architectures have been evaluated for edge deployment with some consistency across studies. Rey et al. [Rey+] showed that nano and small variants can achieve real-time inference on ARM processors, though with a 3–8 percentage-point accuracy penalty relative to the full model. Magalhães et al. [Mag+19] demonstrated that the most appropriate model variant depends on the latency budget: smaller variants are adequate when the target is below 50 ms, but larger models justify cloud offloading when accuracy needs to exceed 95%.

Quantisation and pruning are probably the most commonly applied post-training compression techniques. Begum et al. [Beg+23] used quantised YOLOv4-Tiny in their Borno-Net Bengali Sign Language system and achieved roughly a  $4\times$  size reduction through 8-bit integer quantisation while keeping detection accuracy intact. Lu et al. [Lu+23] went further on microcontrollers, combining 8-bit quantisation with structured filter pruning to reach 184  $\mu$ W power consumption. This demonstrated that SLR is feasible even on battery-powered wearables, although with a smaller gesture vocabulary than what Raspberry Pi-class systems can handle.

Knowledge distillation (transferring learned behaviour from a large “teacher” model to a compact “student”) offers an alternative to brute-force compression. Zhang et al. [Zha+23] combined knowledge distillation with channel pruning and quantisation-aware training on an ARM-device lightweight network, recovering 3–5% of the accuracy that would otherwise be lost by training the compact model from scratch. Mondal et al. [Mon+23] explored Binary CNNs

for a roughly  $32\times$  size reduction using bitwise operations.

A different strategy is to design lightweight architectures from first principles rather than compressing a general-purpose model down. Cheng et al. [Che+] co-designed a gesture recognition model with specific ARM constraints in mind. Saputra et al. [Sap+24] showed that MobileNetV2 reduces parameters by approximately  $9\times$  relative to standard convolutions, making it an effective feature extractor in multi-stage pipelines.

**Synthesis.** The optimisation literature shows clear gains from quantisation, pruning, distillation, and lightweight architectural design, but the evaluations are almost always done in isolation. No study we found directly compares full-pipeline optimisation choices across multiple ASL fingerspelling architectures on Raspberry Pi 5 hardware. That gap motivates the ablation component of SRQ3.

### 2.2.3. Distributed Edge-Cloud Architectures and Task Offloading

*Relevant to: SRQ1 (task distribution and IOT capabilities) and SRQ2 (pre-processing for bandwidth reduction).*

This subsection examines how prior work has split SLR workloads between edge and cloud, what latency-accuracy trade-offs they achieved, and how their findings informed the bandwidth optimisation design in our methodology.

Panopoulos et al. [Pan+25] provide the most directly comparable edge-cloud SLR architecture. They pair a Raspberry Pi 4 (handling video capture, MJPEG encoding, and ZeroTier network transmission) with a remote server running YOLO11X-seg for detection and classification. Dynamic model selection allowed them to reach 99.07% mAP50 at 42.7 ms end-to-end latency. An important takeaway from their work is that the accuracy-latency trade-off can be partly navigated through model tier selection, rather than requiring different hardware.

Magalhães et al. [Mag+19] formalise total inference time across a distributed system as  $T_{\text{total}} = T_{\text{edge}} + T_{\text{net}} + T_{\text{cloud}}$ , where  $T_{\text{net}} = D/b$  ( $D$  = data size,  $b$  = bandwidth). Testing across 3G, 4G, Wi-Fi, and Cable conditions, they showed  $T_{\text{net}}$  dominates at low bandwidths. This result directly motivated the bandwidth optimisation experiment in SRQ2 and the latency decomposition protocol in §3.5.2.

Amanatidis et al. [Ama+23] developed a cooperative task execution framework for IOT object detection that dynamically allocates work across edge nodes and cloud servers based on workload and network conditions. Hadidi et al. [Had+18] demonstrated distributed inference through layer-level partitioning across IOT devices, establishing that communication overhead can be kept within real-time bounds.

Zafar et al. [Zaf+25] and Jin et al. [Jin+25] demonstrated federated learning for edge-based gesture recognition. This thesis does not implement federated learning, but these works confirm it is compatible with Raspberry Pi-class hardware and could be a useful direction for future extensions.

**Synthesis.** Pi-to-GPU-server feasibility has been demonstrated (Panopoulos et al. [Pan+25]), and the bandwidth-latency framework is well-understood (Magalhães et al. [Mag+19]). What has not been done is combining three things: (a) comparing three architectures under identical distributed conditions, (b) quantifying bandwidth as a first-class metric rather than an afterthought, and (c) characterising edge-device resource utilisation on the Pi 5 specifically. Addressing all three is the primary technical contribution of this thesis.

## 3. Methodology

Building on the theoretical foundations and gaps identified in the previous chapter, the next step is to translate these insights into a concrete experimental approach. This chapter defines the methodology, specifying how the proposed system is designed, implemented, and evaluated in order to answer the research questions. It outlines the modelling choices, experimental setup, and evaluation framework that enable a systematic comparison of architectures and a rigorous analysis of their performance across accuracy, latency, bandwidth, and resource utilisation.

The overarching claim we are testing is that an integrated, multi-objective approach (one that simultaneously targets recognition accuracy, end-to-end latency, edge-device resource utilisation, and network bandwidth) can outperform prior single-dimension approaches that optimise for one objective at the cost of others. To move from this general claim to actionable evaluation, we specify three interpretation principles upfront. First, a *meaningful accuracy difference* is defined as  $\geq 2$  percentage points, which exceeds the variability introduced by random seed selection and is detectable with 95% confidence given the test-set size (§ refsec:threats-ch4). Second, *trade-offs between objectives* are evaluated by identifying Pareto-dominant configurations, a pipeline is considered superior only if it improves on at least one metric without degrading any other beyond the stated threshold. Third, *conclusions are drawn* by comparing measured values against the literature-grounded benchmarks in Table 3.3: if a configuration meets the Alsharif et al. [Als+23] accuracy floor and the Panopoulos et al. [Pan+25] latency ceiling simultaneously, the multi-objective claim is supported; if not, the specific bottleneck is identified through the latency decomposition.

### 3.1. Theoretical Framework

This thesis sits at the intersection of three technical sub-fields: deep learning model optimisation for computer vision, IoT edge-device deployment, and distributed edge-cloud systems engineering. The theoretical framework below defines the key constructs, how they are operationalised, and the relationships we expect to find among them.

#### 3.1.1. Key Constructs and Definitions

*ASL fingerspelling recognition* is defined as the task of classifying a static hand configuration, captured in a single video frame, into one of 26 letter classes (A–Z) of the American manual alphabet. The unit of analysis is the individual letter prediction per processed frame. Word-level recognition is treated as a secondary outcome, evaluated through the buffer-based SequenceProcessor already implemented in the system (§3.4.4).

*End-to-end latency* is the elapsed clock time in milliseconds from frame capture on the Pi 5 to receipt of the classification result back on the edge device. We distinguish three additive components: (1)  $T_{\text{edge}}$ —frame capture and JPEG compression; (2)  $T_{\text{net}}$ —transmission from Pi 5 to server, including Wi-Fi and VPN overhead; and (3)  $T_{\text{cloud}}$ —server-side preprocessing, inference, and response. This decomposition follows the same component structure used by

Panopoulos et al. [Pan+25] and makes it possible to pinpoint the dominant bottleneck for each architecture.

*Edge-cloud task distribution* refers to the assignment of pipeline stages to either the Pi 5 or the GPU server. In this thesis, the split is fixed: the Pi 5 handles video capture and JPEG-encoded frame transmission, while all MediaPipe processing, model inference, and sequence processing execute on the server.

*Letter-level classification accuracy* is operationalised as top-1 accuracy on the held-out test set, complemented by per-class macro-averaged F1-score across the 26 letter classes.

*Network bandwidth* is operationalised as the mean data rate (kB/s) from Pi 5 to server, measured at the socket level. Together with end-to-end latency, this directly addresses SRQ2.

*Edge-device resource utilisation* is operationalised as (a) mean CPU utilisation (%) and (b) peak RAM consumption (MB), measured with `psutil` during a 60s steady-state capture window.

### 3.1.2. Hypothesised Relationships

Three pipeline configurations are compared throughout this chapter. The **Baseline** combines MediaPipe hand landmark extraction with an SVM classifier. **Option A** uses a two-stage approach: YOLOv8-pose detects and crops the hand, then EfficientNet-B0 classifies the gesture. **Option B** is an end-to-end YOLOv8 detector trained directly on the 26-letter classes. All three share the same edge-cloud split (Pi 5 for capture and transmission; GPU server for inference). Full specifications are given in §3.2.1.

The following hypotheses are directional expectations that guide the experimental design. They are not formal null-hypothesis tests, we state upfront what we expect to find and why, so that the results section can evaluate whether these expectations held up:

- H1 (Accuracy): Option A (YOLOv8-pose + EfficientNet-B0) will achieve higher letter-level accuracy than the Baseline (MediaPipe + SVM). The reasoning is straightforward: Alsharif et al. [Als+23] report 99.95% accuracy for EfficientNet on the same dataset, which sets the ceiling for this architecture.
- H2 (Latency): Option B (end-to-end YOLOv8) will achieve lower end-to-end latency than Option A, because it eliminates the intermediate crop-and-classify stage. Panopoulos et al. [Pan+25] show that YOLO inference takes 36 ms to 38 ms on a GPU server; removing the second model stage should reduce total latency by a meaningful margin.
- H3 (Resources): All three architectures will show comparable edge-device CPU utilisation and RAM consumption. This is expected because the Pi 5 performs only JPEG capture and transmission in every case, the heavy work is all server-side.
- H4 (Bandwidth): JPEG quality is the primary determinant of bandwidth. Reducing quality from  $Q=95$  to  $Q=70$  should cut bandwidth by at least 40% with less than 3 percentage-point accuracy degradation.

H1 and H2 address SRQ3 and SRQ4. H3 and H4 address SRQ1 and SRQ2 respectively.

| Config   | Edge (Pi 5)    | Server: Preproc.      | Server: Inference        | Primary metric     |
|----------|----------------|-----------------------|--------------------------|--------------------|
| Baseline | Capture + JPEG | MediaPipe Hands (CPU) | SVM on 36-dim features   | Top-1 accuracy, F1 |
| Option A | Capture + JPEG | YOLOv8-pose (GPU)     | EfficientNet-B0          | Top-1 accuracy, F1 |
| Option B | Capture + JPEG | Resize + normalise    | Custom YOLOv8 end-to-end | mAP50, mAP50-95    |

Table 3.1.: Primary experimental configurations.

### 3.1.3. Modelling Perspective

All performance claims in this thesis are associational and predictive, not causal. The controlled design supports internal validity claims about which pipeline performs best within the specific hardware, dataset, and network conditions described here. Generalisation to other settings is treated as an external validity claim, qualified in § refsec:threats-ch4 and in the Discussion.

## 3.2. Research and Experimental Design

The study uses a comparative experimental design: three pipeline configurations evaluated on the same dataset, the same hardware, and the same evaluation protocol. The primary accuracy evaluation is offline, using the held-out test set of static images. System-level evaluation (latency, bandwidth, and resource utilisation) is live, with the full Pi-to-server pipeline running while the researcher fingerspells in front of the camera under real network conditions. No formal user study involving external participants is conducted; all evaluation is performed by the researcher as system operator. The design is prospective: the experimental protocol was defined before implementation began.

Three scope decisions deserve explicit justification. First, all system-level measurements are taken by a single operator (the researcher) rather than by multiple users. This choice is deliberate: the primary research question concerns the technical performance of the pipeline architectures, not user experience or signer variability. A multi-user study would introduce confounds (signing speed, hand size, familiarity with the system) that would obscure the architecture-level comparison. The limitation is that results do not characterise how the system would perform across diverse signers; this is noted as a threat to external validity in § refsec:threats-ch4. Second, the Wi-Fi + VPN network introduces latency variability that is inherent to the deployment scenario. Rather than treating this variability as noise to be eliminated, we report distributional statistics (mean, SD, median, 95th percentile) so that the reader can assess both typical and worst-case performance. Third, the offline accuracy evaluation uses static images, not live video. This means that results generalise directly to the dataset conditions (controlled backgrounds, limited signer diversity) but may not transfer to unconstrained environments, a limitation shared with all published studies using this dataset [Als+23].

### 3.2.1. The Three Pipeline Configurations

Table 3.1 summarises the three configurations. The buffer-based word formation evaluation (§3.2.3) is a secondary analysis and is not part of the primary three-way comparison.

*Baseline: MediaPipe + SVM.* MediaPipe Hands runs on the server CPU and extracts 21 three-dimensional hand landmarks per frame, from which a 36-dimensional feature vector

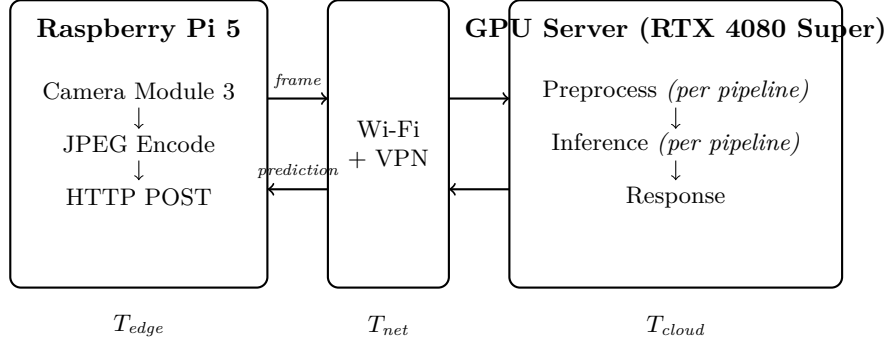


Figure 3.1.: Distributed pipeline architecture. The Raspberry Pi 5 handles capture and transmission ( $T_{\text{edge}}$ ); the GPU server handles preprocessing and inference ( $T_{\text{cloud}}$ ); network transit accounts for  $T_{\text{net}}$ . Preprocessing and inference stages differ per pipeline (Table 3.1).

is computed. This vector consists of: 21 Euclidean wrist-to-landmark distances, 10 normalised fingertip relative positions, and 5 finger bend angles from MCP-PIP-tip triplets. Features are standardised with `StandardScaler` fitted on the training set only. The SVM (`sklearn.svm.SVC`, RBF kernel) is tuned by 5-fold CV over  $C \in \{0.1, 1, 10, 100, 1000\}$  and  $\gamma \in \{\text{'scale'}, \text{'auto'}, 0.001, 0.01, 0.1, 1, 10\}$ . The Baseline also includes a `SequenceProcessor` for buffer-based word formation, evaluated in §3.2.3.

*Option A: YOLOv8-pose + EfficientNet-B0.* A YOLOv8n-pose model detects the hand and extracts 21 two-dimensional keypoints in a single GPU forward pass. The bounding box is expanded by 10%, cropped, and resized to  $224 \times 224$  for EfficientNet-B0 input. EfficientNet-B0 (5.3 M parameters, ImageNet pretrained) is fine-tuned with its classification head replaced by a  $1280 \rightarrow 26$  linear layer with dropout ( $p=0.2$ ).

*Option B: Custom YOLOv8 end-to-end.* A YOLOv8n model (3.2 M parameters) is fine-tuned from COCO weights to directly output 26 ASL letter class predictions alongside bounding boxes in a single forward pass. The idea here is to target the lowest end-to-end latency of the three configurations by eliminating the two-stage overhead.

### 3.2.2. Network Architecture and the Edge-Cloud Split

All three configurations assign the same responsibilities to the Pi 5: frame capture, JPEG encoding, and HTTP POST transmission via a Flask REST API. The server handles everything compute-intensive. The actual network path is: Pi 5 connects via Wi-Fi to the local router, which connects through a VPN tunnel to the university network where the GPU server sits.  $T_{\text{net}}$  therefore includes both Wi-Fi latency and VPN overhead, this is the real deployment condition, not a laboratory idealisation.

We made a deliberate decision not to treat Wi-Fi variability as a confound to be eliminated. It is a real characteristic of the deployment scenario described in the Introduction, a portable Pi used wherever there is internet access. Latency and bandwidth experiments are conducted over this actual topology, and the results reflect what a user would actually experience.

Three additional justifications support the fixed split. First, it isolates model effects: the edge-cloud boundary is held constant, so any performance differences are attributable to the server-side architecture. Second, there is an operational stability issue, during early testing, we found that MediaPipe causes GPU context instability on the Pi 5. Third, Panopoulos

et al. [Pan+25] use the same Pi-to-GPU-server split, which makes the results architecturally comparable.

### 3.2.3. Word Formation Evaluation

The Baseline pipeline includes a buffer-based SequenceProcessor that forms words from sequences of letter predictions. This component is already implemented; we evaluate it here as a secondary analysis using two complementary metrics: Word Accuracy (primary) and Character Error Rate (secondary/diagnostic).

**Rationale for two complementary metrics.** Letter-level accuracy measures whether each individual frame is correctly classified. Word-level accuracy measures whether the system produces a correct word, which is ultimately the user-facing objective of fingerspelling. These two metrics capture different failure modes and are not interchangeable. To make the relationship concrete: if letter-level accuracy is  $p$ , the expected probability that an  $n$ -letter word is spelled completely correctly is approximately  $p^n$  assuming errors are independent across frames. For  $p = 0.90$  and a 5-letter word, expected word accuracy is  $0.90^5 \approx 59\%$ ; for a 7-letter word it drops to  $0.90^7 \approx 48\%$ . Even seemingly high letter accuracy can produce surprisingly low word accuracy without temporal smoothing. Reporting both metrics reveals whether the SequenceProcessor successfully closes this gap. This framing is consistent with Saputra et al. [Sap+24], who separately report inference speed, word error rate, and sentence accuracy, showing that sequence-level outcome metrics add information beyond what frame-level accuracy provides.

Both metrics are defined formally in §3.5: Word Accuracy is the proportion of words produced completely correctly (primary), and CER measures the character-level edit distance (secondary/diagnostic).

**Evaluation procedure.** The vocabulary of 50 words consists of common English nouns, verbs, and adjectives (no proper nouns, abbreviations, or technical terms), ranging from 4 to 6 letters in length (24 four-letter, 25 five-letter, and one six-letter word; mean 4.54), with every letter of the alphabet appearing at least once. Both metrics are computed over all 50 words. The evaluation has two stages:

- *Synthetic evaluation (primary):* For each word, we construct a synthetic sequence of frames by selecting one test-set image per letter in word order. “HELLO” becomes five frames: H, E, L, L, O. This sequence is fed through the full Baseline pipeline including the SequenceProcessor. Using static test-set images makes this evaluation fully reproducible across runs. Both Word Accuracy and CER are computed from the 50 synthetic sequences.
- *Live demonstration (secondary):* The researcher fingerspells the same 50 words in front of the camera. This captures hand-transition effects and natural pacing variation that synthetic sequences cannot simulate. Word Accuracy and CER are reported descriptively alongside the synthetic results, providing a qualitative picture of real-world performance.

### 3.2.4. Factors, Conditions, and Controls

*Independent variable:* Pipeline configuration (Baseline, Option A, Option B). All other factors are held constant:

- Hardware: Pi 5 (8 GB RAM, ARM Cortex-A76 @ 2,4 GHz); RTX 4080 Super server (16 GB VRAM, CUDA 12.2).
- Dataset: Same 70/15/15 stratified split (seed = 42) for all configurations.
- Network: Wi-Fi to local router + VPN to university server. All measurements taken over this real topology.
- JPEG quality:  $Q=85$  for primary evaluations; secondary experiment sweeps  $Q \in \{50, 70, 85, 95\}$  and frame rates  $\in \{10, 15, 20, 30\}$  fps.
- Frame resolution:  $640 \times 480$  on Pi; resized per model requirements on server.

*Dependent variables:* (1) letter-level top-1 accuracy, (2) per-class macro-averaged F1-score, (3) mAP50 and mAP50-95 for Option B (Option B uses detection metrics rather than top-1 accuracy because it is a detection model whose output is bounding boxes with class labels rather than a classification probability distribution; all other evaluation conditions are identical), (4) end-to-end latency (mean, SD, median, 95th percentile) with  $T_{\text{edge}}/T_{\text{net}}/T_{\text{cloud}}$  decomposition, (5) edge CPU utilisation (%), (6) edge RAM consumption (MB), (7) network bandwidth (kB/s). For the Baseline word formation evaluation only: (8) Word Accuracy (primary) and (9) CER (secondary/diagnostic).

### 3.3. Data Collection and Preprocessing

#### 3.3.1. Dataset

This study uses the ASL Alphabet dataset (Kaggle user “grassknotted”, <https://www.kaggle.com/datasets/grassknotted/asl-alphabet>), comprising 87,000 RGB images at  $200 \times 200$  pixels across 29 classes. We excluded the three auxiliary classes (*space*, *delete*, *nothing*), yielding 78,000 images across 26 letter classes A–Z with exactly 3,000 images per class. Alsharif et al. [Als+23] used this same dataset to establish the model-level classification benchmark, which is one of the main reasons we selected it, it enables direct comparison.

**Known limitations.** There are four that need to be acknowledged upfront. (1) *Static images only*, the dataset contains still images, not video; the word-formation evaluation uses synthetic sequences as described in §3.2.3. (2) *Letters J and Z* involve motion in authentic ASL; static snapshots may underestimate performance on naturally signed versions, so these are reported separately. (3) *No subject identifiers*, signer identity is not recorded in the dataset, which means subject-wise splitting is impossible. The train and test sets likely share signers, which could make reported accuracy slightly optimistic for unseen signers. This is a limitation shared by Alsharif et al. [Als+23] and most published studies using this dataset. (4) *Controlled conditions*, performance under cluttered backgrounds or outdoor lighting is not characterised.

**Dataset split.** 78,000 images partitioned into training (70%), validation (15%), test (15%) via single stratified random split (seed = 42). Applied once and held constant across all evaluations. Table 3.2 summarises the partition.

| Subset     | Images (total) | Images per class | Purpose                               |
|------------|----------------|------------------|---------------------------------------|
| Training   | 54 600         | 2100             | Model parameter optimisation          |
| Validation | 11 700         | 450              | Hyperparameter tuning, early stopping |
| Test       | 11 700         | 450              | Final unbiased performance evaluation |

Table 3.2.: Dataset partition summary.

### 3.3.2. Preprocessing Pipeline

The preprocessing pipeline applies five steps in order:

1. *Format conversion*: BGR to RGB via `cv2.cvtColor(image, cv2.COLOR_BGR2RGB)` immediately after loading.
2. *Resize*:  $640 \times 640$  for YOLOv8;  $224 \times 224$  for EfficientNet-B0; resolution-agnostic for the SVM landmark pipeline. All resizing uses bilinear interpolation (`cv2.INTER_LINEAR`).
3. *Normalisation*:  $[0, 1]$  scaling; ImageNet channel-wise normalisation ( $\mu = [0.485, 0.456, 0.406]$ ,  $\sigma = [0.229, 0.224, 0.225]$ ) for EfficientNet-B0. Statistics fitted on the training set only.
4. *Data augmentation (training only)*: Random rotation  $\pm 15^\circ$ ; brightness/contrast jitter  $\pm 20\%$ ; translation  $\pm 10\%$ ; scaling  $0.9\text{--}1.1\times$ . Horizontal flip is unconditionally disabled, handedness carries semantic meaning in ASL, so flipping would create incorrect training examples.
5. *Label encoding*: A–Z mapped to indices 0–25. YOLO format (normalised centre- $x$ , centre- $y$ , width, height) for bounding boxes.

## 3.4. Methods and Analysis Procedure

### 3.4.1. Baseline: MediaPipe + svm

MediaPipe Hands extracts 21 3D landmarks per frame on the server CPU, producing a 63-element raw array from which the 36-dimensional feature vector is computed (as described in §3.2.1). `StandardScaler` is fitted on the training set only. The SVM is tuned by 5-fold CV over the same  $C$  and  $\gamma$  grids specified above. The final model is trained on the full training set and evaluated once on the held-out test set.

### 3.4.2. Option A: YOLOv8-pose + EfficientNet-B0

YOLOv8n-pose detects the hand and extracts 21 two-dimensional keypoints in a single GPU forward pass. The bounding box is expanded by 10%, cropped, and resized to  $224 \times 224$ . EfficientNet-B0 is then fine-tuned in two phases:

- *Phase 1 (5 epochs)*: Backbone frozen; only the new  $1280 \rightarrow 26$  classification head is trained. AdamW optimiser,  $\text{lr} = 1 \times 10^{-3}$ , weight decay  $= 1 \times 10^{-4}$ .
- *Phase 2 (up to 45 epochs)*: All layers unfrozen. AdamW,  $\text{lr} = 1 \times 10^{-4}$ , cosine annealing to  $\text{lr} = 1 \times 10^{-6}$ . Early stopping with patience = 5 on validation accuracy. Batch size = 32.

Loss function is categorical cross-entropy. The model is deployed as a Flask REST API; the Pi 5 transmits JPEG frames via HTTP POST and receives the predicted letter, confidence, and timestamp in the response.

### 3.4.3. Option B: Custom YOLOv8 End-to-End

YOLOv8n is fine-tuned from COCO weights using the SGD optimiser with initial  $lr = 0.01$  and a cosine LR schedule, for up to 100 epochs with early stopping at  $patience = 20$  on validation mAP50 via the Ultralytics `train()` API. Mosaic augmentation, mixup, and horizontal flip are all disabled, the flip for the same ASL handedness reason, and the others because they distort the hand shape in ways that hurt performance on this specific task.

### 3.4.4. Buffer-Based Word Formation (Baseline)

The Baseline pipeline includes a `SequenceProcessor` that accumulates consecutive letter predictions and applies majority-vote smoothing to form words. A letter is confirmed if: (a) it appears as the majority prediction across the last  $N$  frames ( $N=3$ , configurable 3–5), and (b) it has held that position for at least 200 ms. Word boundaries are detected by a  $> 1$  s pause with no hand detected in frame, a threshold chosen to be  $3\text{--}5\times$  longer than typical inter-letter transitions ( $\sim 100\text{--}300$  ms), ensuring robustness to brief hand occlusions between letters while remaining within the natural inter-word pause of a non-expert signer.

The `SequenceProcessor` operates purely on the temporal pattern of classifier outputs. It uses no language model, no dictionary, nothing beyond the raw predictions. Systematic letter-level errors therefore propagate directly into word output. This behaviour is expected, and it is exactly why the word-level evaluation in §3.2.3 matters: the gap between letter-level accuracy and Word Accuracy quantifies the practical impact of per-letter errors on real communication.

### 3.4.5. Training Infrastructure

All training was performed on: AMD Ryzen 9, NVIDIA RTX 4080 Super (16 GB VRAM), 64 GB RAM, Ubuntu 22.04, CUDA 12.2, PyTorch 2.x, Ultralytics 8.x, scikit-learn 1.x. The Pi 5 (ARM Cortex-A76 @ 2.4 GHz, 8 GB LPDDR4X, Raspberry Pi OS 64-bit) is used exclusively for inference experiments. Random seed 42 is set globally.

## 3.5. Evaluation Procedure and Metrics

The evaluation metrics are organised into two formally distinct categories. *Model-level metrics* assess the intrinsic performance of the recognition algorithm and are grounded in Alsharif et al. [Als+23]. *System-level metrics* assess the complete distributed pipeline and are grounded in Panopoulos et al. [Pan+25].

To ensure that numerical results translate into actionable conclusions, each metric is paired with an interpretation threshold that distinguishes adequate from inadequate performance in the context of this thesis. For model-level accuracy, any configuration achieving  $\geq 97\%$  top-1 accuracy (within 3 percentage points of the Alsharif et al. ceiling) is considered adequate for practical fingerspelling; performance below 90% would indicate a fundamental architectural limitation. For end-to-end latency,  $\leq 66$  ms ( $\geq 15$  fps) is the real-time threshold grounded in Panopoulos et al. [Pan+25]; latency above 200 ms would render the system unsuitable for

interactive use. For Word Accuracy, because no prior baseline exists on this specific system, we interpret results against the theoretical  $p^n$  floor (e.g.,  $0.90^5 \approx 59\%$  for 5-letter words at 90% letter accuracy), any result substantially above this floor indicates that the SequenceProcessor adds practical value. These thresholds are not formal pass/fail criteria but interpretive anchors that connect the metrics to real-world usability: a system that meets all three simultaneously would be suitable for deployment as a communication aid, while one that fails on latency but succeeds on accuracy would require hardware or network improvements before deployment.

### 3.5.1. Model-Level Metrics

Model-level metrics are evaluated on the held-out test set ( $\sim 11,700$  frames) after all hyperparameter selection has been completed on the validation set. Because Baseline and Option A are pure classification tasks while Option B is a detection task, two different metric sets apply.

*For Baseline (SVM) and Option A (EfficientNet-B0), following Alsharif et al. [Als+23]:*

- Top-1 accuracy =  $(TP + TN)/(TP + TN + FP + FN)$ . Primary metric. Comparison: EfficientNet 99,95%, ResNet-50 99,98%.
- Precision =  $TP/(TP + FP)$ . Comparison: EfficientNet 99,90%.
- Recall =  $TP/(TP + FN)$ . Comparison: EfficientNet 99,94%.
- Macro-averaged F1 =  $2 \times (P \times R)/(P + R)$ , equally weighted across 26 classes. Comparison: EfficientNet 99,92%.
- Per-class confusion matrix ( $26 \times 26$ ), shown in Figures 5.1–5.3 (Section 5.1) and summarised in Appendix A. Essential for identifying confusable letter pairs like A/E/S or M/N/T.

*For Option B (YOLOv8 detection task), following Panopoulos et al. [Pan+25]:*

- mAP50—mean AP at IoU  $\geq 0.50$  across 26 classes. Comparison: 99,07%.
- mAP50-95—mean AP at IoU 0.50–0.95. Comparison: 87,7%.

*Word formation metrics (Baseline only).* In addition to the frame-level metrics, the Baseline word formation capability is evaluated using two sequence-level metrics, measured on the synthetic 50-word vocabulary. The motivation for using both a binary outcome metric and a degree-of-error metric comes from Saputra et al. [Sap+24], who separately report sentence accuracy and word error rate for their sign language pipeline.

*Word Accuracy* is defined as the proportion of words in the test vocabulary that the system produces completely correctly:

$$\text{Word Accuracy} = \frac{\text{number of correctly spelled words}}{\text{total words in test vocabulary}}$$

A word counts as correct if and only if every character of the output matches the reference. Word Accuracy = 1.0 means all 50 test words were spelled without error; Word Accuracy = 0.0 means none were. This is the primary metric because it directly answers the user-facing question: did the system produce the correct word?

*Character Error Rate (CER)* is defined as:

$$\text{CER} = \frac{S + D + I}{N}$$

where  $S$ ,  $D$ , and  $I$  are character-level substitutions, deletions, and insertions, and  $N$  is the total reference character count. CER is the diagnostic metric: it explains how much is wrong when Word Accuracy fails. A word with  $\text{CER} = 0.20$  has one wrong character in every five; a word with  $\text{CER} = 0.80$  is largely unrecognisable. Together, Word Accuracy tells us if the system worked; CER tells us how badly it failed when it did not.

Each metric family has strengths and limitations that should be made explicit. Top-1 accuracy is intuitive and directly comparable to Alsharif et al. [Als+23], but it can mask per-class weaknesses on confusable letters (e.g., M/N/T), hence the complementary confusion matrix. Macro-averaged F1 treats all 26 classes equally regardless of frequency, which is appropriate here because the dataset is balanced, but would require adjustment on imbalanced data. mAP50 and mAP50-95 are standard for detection tasks and enable comparison with Panopoulos et al. [Pan+25], but they conflate localisation and classification quality, a high mAP50 does not guarantee that every letter is correctly classified if bounding boxes are accurate but class labels are wrong. Word Accuracy captures the user-facing outcome (was the word correct?) but is a binary metric that loses information about partial correctness, this is why CER accompanies it as a diagnostic. End-to-end latency is the most deployment-relevant system metric, but it is sensitive to network conditions; the 95th percentile and latency decomposition address this by separating controllable (model choice) from uncontrollable (network variability) factors.

### 3.5.2. System-Level Metrics

System-level metrics are measured on the live  $\text{Pi } 5 \rightarrow \text{Wi-Fi} \rightarrow \text{VPN} \rightarrow \text{server}$  pipeline. All measurements reflect the real deployment topology described in §3.2.2.

*End-to-end latency* (ms) is the primary system metric. Measured with `time.perf_counter()` over 300 steady-state frames and reported as mean, SD, median, and 95th percentile. Decomposed into  $T_{\text{edge}}$ ,  $T_{\text{net}}$ ,  $T_{\text{cloud}}$  following the same component structure as Panopoulos et al. [Pan+25].

*Real-time threshold*:  $\geq 15$  fps ( $\leq 66$  ms/frame), operationally confirmed by Panopoulos et al. [Pan+25] who achieve 23,4fps on a comparable architecture. Worth noting: their 42,7ms figure is server-side only. This thesis measures true end-to-end latency including Wi-Fi and VPN overhead, so the numbers are not directly comparable.

*Server-side inference time* (ms) is measured with `torch.cuda.Event` to isolate  $T_{\text{cloud}}$  and identify bottlenecks within the server-side pipeline.

*Network bandwidth* (kB/s) socket-level byte counts divided by transmission time. Includes Wi-Fi and VPN overhead. Directly addresses SRQ2.

*Edge CPU utilisation (%)* and *RAM consumption* (MB) are sampled at 1 Hz over 60s via `psutil`. Mean and peak values are reported. These are novel measurements, there is no prior baseline for Pi 5 in an ASL edge-cloud context.

Table 3.3 provides the complete metric summary.

| Metric                | Category | Applies to       | Grounded in                | Comparison                                               |
|-----------------------|----------|------------------|----------------------------|----------------------------------------------------------|
| Top-1 accuracy        | Model    | Baseline, Opt. A | Alsharif et al. [Als+23]   | 99.95%                                                   |
| Precision             | Model    | Baseline, Opt. A | Alsharif et al. [Als+23]   | 99.90%                                                   |
| Recall                | Model    | Baseline, Opt. A | Alsharif et al. [Als+23]   | 99.94%                                                   |
| Macro-F1              | Model    | Baseline, Opt. A | Alsharif et al. [Als+23]   | 99.92%                                                   |
| mAP50                 | Model    | Opt. B only      | Panopoulos et al. [Pan+25] | 99.07%                                                   |
| mAP50-95              | Model    | Opt. B only      | Panopoulos et al. [Pan+25] | 87.7%                                                    |
| Confusion matrix      | Model    | All three        | Both papers                | Qualitative analysis                                     |
| Word Accuracy         | Model    | Baseline only    | Saputra et al. [Sap+24]    | No prior baseline                                        |
| CER                   | Model    | Baseline only    | Saputra et al. [Sap+24]    | WER: 16.42%                                              |
| E2E latency           | System   | All three        | Panopoulos et al. [Pan+25] | et 42,7 ms (server ref.)                                 |
| Latency decomposition | System   | All three        | Panopoulos et al. [Pan+25] | et $T_{\text{edge}} + T_{\text{net}} + T_{\text{cloud}}$ |
| Real-time threshold   | System   | All three        | Panopoulos et al. [Pan+25] | et $\leq 66$ ms ( $\geq 15$ fps)                         |
| Bandwidth             | System   | All three        | No direct baseline         | Novel                                                    |
| Edge CPU              | System   | Pi 5 only        | No prior baseline          | Novel                                                    |
| Edge RAM              | System   | Pi 5 only        | No prior baseline          | Novel                                                    |

Table 3.3.: Complete metric summary with literature grounding.

### 3.5.3. Secondary Experiment: Bandwidth–Accuracy Trade-off (srq2)

Using the best-performing base architecture, a secondary  $4 \times 4$  factorial experiment (no retraining) evaluates JPEG quality  $Q \in \{50, 70, 85, 95\}$  crossed with frame rate  $\in \{10, 15, 20, 30\}$  fps. All 16 conditions are measured over the real Wi-Fi + VPN topology. Results will be presented as a Pareto surface of accuracy vs. bandwidth vs. latency. The goal is to find the  $Q$  and fps combination that achieves  $\leq 66$  ms latency and  $\geq 15$  fps (the Panopoulos et al. [Pan+25] threshold) at minimum bandwidth while staying within 3 percentage points of the Alsharif et al. offline benchmark [Als+23]. This experiment directly answers SRQ2.

## 3.6. Reliability and Validity

### 3.6.1. Reliability

*Pipeline determinism.* Seed 42 is applied globally. All deep learning experiments are repeated with seeds  $\{42, 0, 123\}$ ; performance is reported as mean  $\pm$  SD.

*Measurement reliability.* Latency measurements use 300 steady-state frames. Resource measurements run for 60 s at 1 Hz. The Wi-Fi + VPN link introduces latency variability that is inherent to the deployment scenario; we report the 95th percentile to characterise tail behaviour honestly. All measurements are taken during normal working hours without network isolation again, reflecting real conditions.

*Computational reproducibility.* The full environment is documented in `requirements.txt`, `environment.yml`, and a `README.md` runbook. Trained weights and evaluation scripts are

deposited in the thesis GitHub repository.

### **3.6.2. Threats to Validity**

A full structured threats-to-validity analysis covering construct validity (metric–construct alignment), internal validity (leakage prevention, fair baseline tuning), external validity (dataset homogeneity, network topology specificity), and statistical conclusion validity (bootstrap confidence intervals, Bonferroni correction for three pairwise comparisons), together with a residual risk statement is presented in Section 4.5 (Chapter 4), where it is positioned alongside the implementation details it qualifies. The placement there follows the guideline that threats should be documented in the Experimental Setup once the concrete design decisions are known; the current section provides the reliability assurances that are design-level commitments, independent of implementation.

## 4. Experimental Setup

This chapter serves two functions. First, it provides enough implementation detail for an independent researcher to replicate the experiments using the code deposited at the thesis repository (commit `c9e2aa3`). Second, it characterises the dataset through exploratory analysis so that the reader can assess how representative the experimental conditions are. Design-level decisions (pipeline configurations, evaluation protocol, and the rationale for each choice) are described in Chapter 3; this chapter reports what was actually instantiated.

### 4.1. Dataset Description and Exploratory Analysis

The dataset is introduced in §3.3.1. This section adds the descriptive statistics and exploratory plots that allow the reader to assess data quality and anticipate potential failure modes.

#### 4.1.1. Class Balance

The ASL Alphabet dataset is perfectly balanced by design: each of the 26 letter classes contains exactly 3,000 images, yielding 78,000 images in total after the three auxiliary classes are excluded. After the stratified 70/15/15 split (seed = 42), each subset contains 2,100 (training), 450 (validation), and 450 (test) images per class with no remainder. Perfect balance means class-weighted loss is not required for the primary accuracy evaluation; however, we apply class-weighted cross-entropy for EfficientNet-B0 and class-weighted SGD for YOLOv8 because MediaPipe detection failures (described below) introduce an effective imbalance in the annotated subsets seen during training.

#### 4.1.2. MediaPipe Detection Recall by Class

Before training Options A and B, all training and validation images were processed by MediaPipe Hands to generate hand landmarks and bounding boxes for the YOLO annotation format. MediaPipe failed to detect a hand in 14 984 of the 66 300 training+validation images (22,6%), and these images were excluded from the annotated subsets used to train YOLOv8-pose and EfficientNet-B0. This failure rate is not uniform across classes (Figure 4.1).

The low detection rates for N and M are explained by their hand configurations: both involve all four fingers flexed or occluded, producing a compact shape that MediaPipe’s landmark model struggles to localise. F and K, by contrast, have clear finger separation. This failure pattern introduces an effective class imbalance of up to  $2.3\times$  between the best-detected letter (F: 2013 annotated training images) and the worst (N: 886). Options A and B therefore apply class-weighted loss; the Baseline SVM is unaffected because it trains on all 54 600 training images regardless of detection outcome.

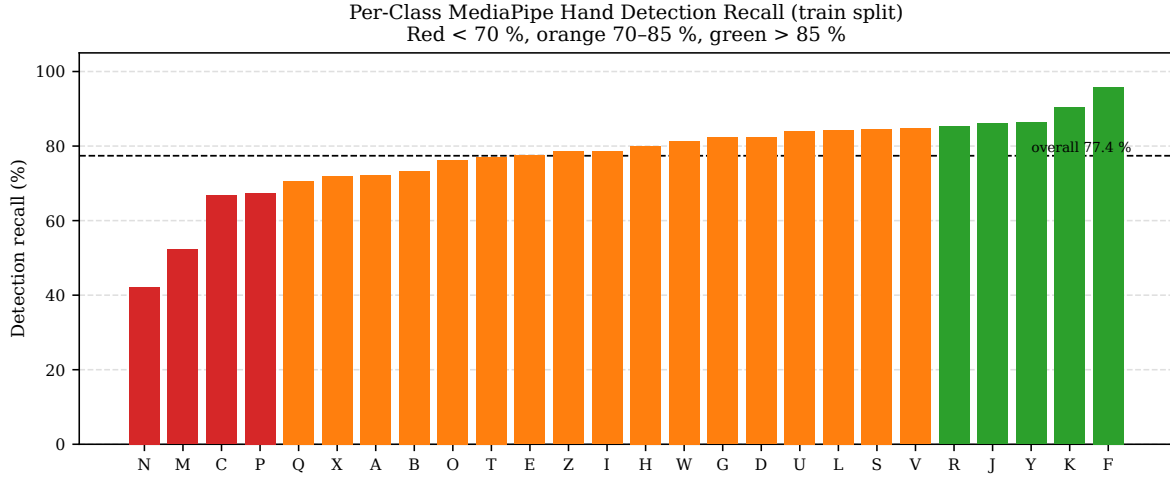


Figure 4.1.: Per-class MediaPipe hand detection recall on the training split, sorted by recall in ascending order. Letters N (42%) and M (52%) are detected far less reliably than F (96%) and K (90%). Red bars indicate recall < 70%, orange 70–85%, green > 85%. The dashed line marks the overall recall of 77,4%. This class-level imbalance in the annotated training data directly affects the effective training set sizes for Options A and B.

#### 4.1.3. Representative Samples

Figure 4.2 shows one randomly selected image per letter (seed = 42). All images share the same controlled acquisition conditions: uniform background, single signer, consistent lighting. This homogeneity is the main external-validity limitation of the dataset (discussed further in §4.5).

## 4.2. Implementation Environment

### 4.2.1. Hardware

Table 4.1 lists the hardware used in all experiments. The Pi 5 runs no inference; its role is limited to JPEG encoding and HTTP POST.

| Component    | Server                             | Edge device                            |
|--------------|------------------------------------|----------------------------------------|
| Role         | Inference + API                    | Capture + transmit                     |
| CPU          | Intel Core i9 (14-core)            | Arm Cortex-A76 @ 2,4 GHz               |
| RAM          | 64 GB DDR5                         | 8 GB LPDDR4X                           |
| GPU / Accel. | NVIDIA RTX 4080 Super (16 GB VRAM) | —                                      |
| CUDA         | 12.4                               | —                                      |
| Storage      | NVMe SSD                           | microSD (Class 10)                     |
| OS           | Ubuntu 22.04 LTS                   | Raspberry Pi OS Bookworm 64-bit        |
| Camera       | —                                  | Logitech C922 Pro (640×480 px, 30 fps) |
| Network      | Uni LAN (1 Gbit/s)                 | Wi-Fi + VPN tunnel                     |

Table 4.1.: Hardware configuration. All experiments use this fixed topology; the hardware is not varied across pipelines.

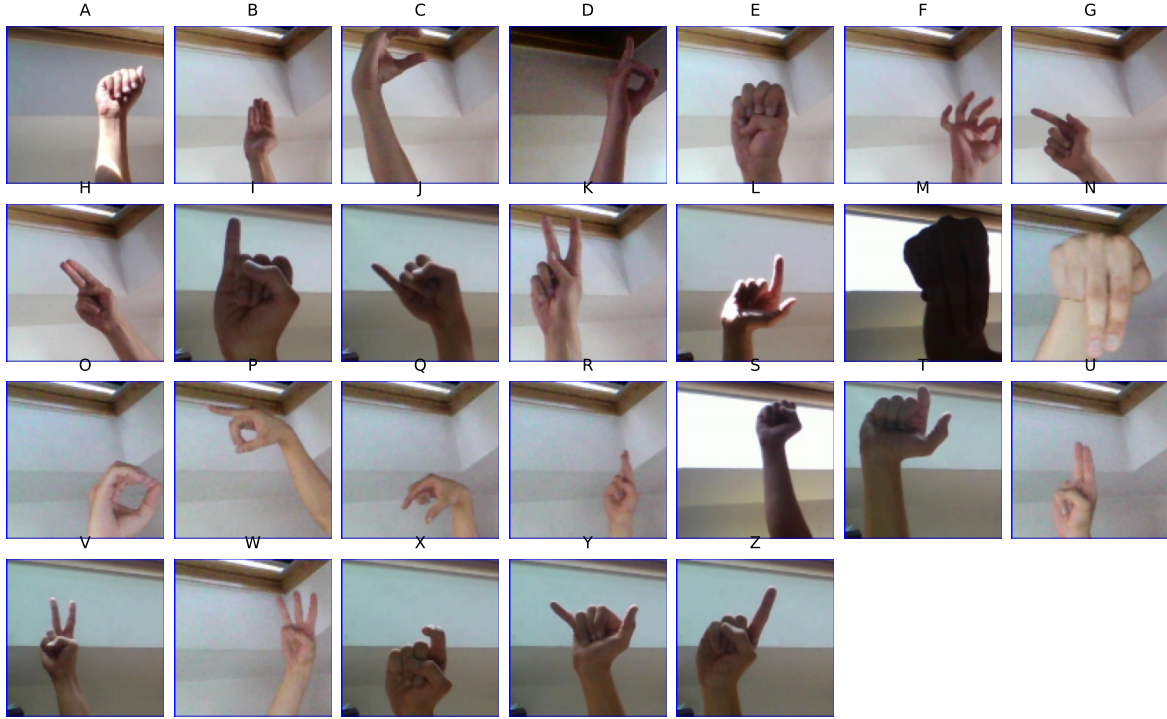


Figure 4.2.: One randomly selected training image per letter (seed = 42), showing the controlled background and consistent hand positioning that characterise the dataset.

#### 4.2.2. Software Versions

Table 4.2 lists the exact library versions used on the server. The Pi 5 runs Python 3.11.2 with `opencv-python 4.x`, `requests 2.x`, and `jiwer 3.x` installed in a `virtualenv`. Full dependency lockfiles are provided in `requirements.txt` (server) and `requirements-pi.txt` (edge device).

### 4.3. Hyperparameter Configuration

All hyperparameters were fixed before any evaluation was performed on the test set. Tuning used the validation set only. The same configuration was applied for seeds  $\{42, 0, 123\}$  except where noted.

#### 4.3.1. Baseline: SVM

Table 4.3 details the kernel choice, the full grid-search space, and the best parameters found for seed 42.

#### 4.3.2. Option A: YOLOv8n-pose (Hand Detection Stage)

Table 4.4 lists the training settings for the detection stage of Option A. This model runs on the server GPU and provides the hand bounding box and 21 keypoints consumed by the EfficientNet-B0 classifier.

| Library        | Version     | Role                               |
|----------------|-------------|------------------------------------|
| Python         | 3.10.12     | Server runtime                     |
| PyTorch        | 2.6.0+cu124 | Deep learning (Options A, B)       |
| Ultralytics    | 8.4.46      | YOLOv8 training and inference      |
| MediaPipe      | 0.10.14     | Hand landmark detection (Baseline) |
| scikit-learn   | 1.7.0       | SVM training and evaluation        |
| OpenCV         | 4.13.0      | Image I/O, JPEG encode/decode      |
| albumentations | 2.0.8       | Training augmentation              |
| Flask          | 3.x         | REST API server                    |
| jiwer          | 3.x         | CER / WER calculation              |
| NumPy          | 1.x         | Array operations                   |

Table 4.2.: Server-side software versions used in all experiments. The git commit `c9e2aa3` (tag `v1.0-thesis-submission`) captures the exact code state that produced all reported results.

| Setting                 | Value                                                                 |
|-------------------------|-----------------------------------------------------------------------|
| Kernel                  | RBF ( <code>rbf</code> )                                              |
| C search grid           | <code>{0.1, 1, 10, 100, 1000}</code>                                  |
| $\gamma$ search grid    | <code>{scale, auto, 0.001, 0.01, 0.1, 1, 10}</code>                   |
| Grid-search CV          | 5-fold on stratified 20% subsample of training set                    |
| Final training          | Full training set (54 600 images) with best $(\hat{C}, \hat{\gamma})$ |
| Best params (seed 42)   | $C = 100, \gamma = 0.01$                                              |
| Best CV score (seed 42) | 98,57 %                                                               |

Table 4.3.: Baseline SVM hyperparameter grid and selected parameters for seed 42. Grid search is run independently for each seed; full per-seed results are in `results/baseline/seed_{42,0,123}/train_manifest.json`.

| Setting               | Value                                                                                               |
|-----------------------|-----------------------------------------------------------------------------------------------------|
| Base model            | YOLOv8n-pose (pretrained on COCO-Pose)                                                              |
| Optimiser             | SGD, momentum 0.937, weight decay $5 \times 10^{-4}$                                                |
| Initial LR ( $lr_0$ ) | 0.01                                                                                                |
| LR schedule           | Cosine annealing to $lr_f = lr_0 \times 0.01$                                                       |
| Epochs                | 50, early-stopping patience 10                                                                      |
| Batch size            | 16                                                                                                  |
| Input size            | $640 \times 640$                                                                                    |
| Keypoints             | 21 hand landmarks ( $x, y$ , visibility)                                                            |
| Augmentation          | Rotation, scale, colour jitter; <code>flipr=0</code> , <code>mosaic=0</code> , <code>mixup=0</code> |
| Seeds                 | <code>{42, 0, 123}</code> ( <code>deterministic=True</code> )                                       |
| Training time         | $\approx 2,1$ h per seed (RTX 4080 Super)                                                           |

Table 4.4.: YOLOv8n-pose training configuration (Option A, detection stage). Results: pose mAP50  $\geq 0.963$  for all seeds (see §5).

### 4.3.3. Option A: EfficientNet-B0 (Classification Stage)

EfficientNet-B0 is fine-tuned in two sequential phases on  $224 \times 224$  crops extracted by the YOLOv8-pose bounding boxes.

| Setting                 | Phase 1 (head only)                                                                                                          | Phase 2 (full fine-tune)                  |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| Layers trained          | Classification head                                                                                                          | All layers                                |
| Optimiser               | AdamW                                                                                                                        | AdamW                                     |
| Learning rate           | $10^{-3}$                                                                                                                    | $10^{-4}$ (cosine $\rightarrow 10^{-6}$ ) |
| Weight decay            | $10^{-4}$                                                                                                                    | $10^{-4}$                                 |
| Max epochs              | 5                                                                                                                            | 45                                        |
| Early stopping patience | —                                                                                                                            | 5 (val accuracy)                          |
| Batch size              | 32                                                                                                                           | 32                                        |
| Loss                    | Class-weighted cross-entropy                                                                                                 |                                           |
| Augmentation            | Rotation $\pm 15^\circ$ ; brightness/contrast $\pm 20\%$ ; translation $\pm 10\%$ ; scale 0.9–1.1 $\times$ ; horizontal flip |                                           |
| Training time           | $\approx 30$ min per seed (RTX 4080 Super)                                                                                   |                                           |

Table 4.5.: EfficientNet-B0 two-phase fine-tuning configuration (Option A, classification stage). Results: val top-1  $\geq 99.92\%$  for all seeds.

### 4.3.4. Option B: YOLOv8n Detect (End-to-End)

Table 4.6 lists the training settings for Option B. Unlike Option A, this single model simultaneously localises the hand and classifies the letter in one forward pass, with no separate classification stage.

| Setting               | Value                                                                                     |
|-----------------------|-------------------------------------------------------------------------------------------|
| Base model            | YOLOv8n-detect (pretrained on COCO)                                                       |
| Number of classes     | 26 (letters A–Z)                                                                          |
| Optimiser             | SGD, momentum 0.937, weight decay $5 \times 10^{-4}$                                      |
| Initial LR ( $lr_0$ ) | 0.01                                                                                      |
| LR schedule           | Cosine annealing                                                                          |
| Epochs                | 100, early-stopping patience 20                                                           |
| Batch size            | 16                                                                                        |
| Input size            | $640 \times 640$                                                                          |
| Augmentation          | Standard geometric/colour; $\text{fliplr} = 0$ , $\text{mosaic} = 0$ , $\text{mixup} = 0$ |
| Seeds                 | {42, 0, 123} ( <b>deterministic=True</b> )                                                |
| Training time         | $\approx 2.9$ h per seed (RTX 4080 Super)                                                 |

Table 4.6.: YOLOv8n-detect training configuration (Option B). Mosaic and mixup are disabled because they corrupt hand-shape semantics. Results: mAP50  $\geq 0.991$  for all seeds.

## 4.4. Live Evaluation Protocol

The offline accuracy evaluation (§3.5) uses static images and runs entirely on the server. The live evaluation additionally exercises the Pi-to-server pipeline under real network conditions and assesses word-formation accuracy with a human signer.

#### 4.4.1. Network Topology

The edge device (Pi 5) connects to the server via Wi-Fi ( $\rightarrow$  local router  $\rightarrow$  openfortivpn tunnel  $\rightarrow$  university LAN  $\rightarrow$  server). The VPN MTU is set to 1354 B (path-MTU probed before each session) to avoid packet fragmentation of the JPEG payloads. The server IP is 10.4.52.10; the Pi VPN address is 10.6.140.9. All latency measurements were recorded during normal working-hours use of the university network, reflecting a realistic deployment scenario rather than an isolated laboratory network.

#### 4.4.2. End-to-End Latency Benchmark

Three 300-second steady-state sessions are recorded per pipeline (one per seed), yielding approximately 3400 frames per session at the camera’s sustained frame rate. The first 30 frames are discarded as warm-up; a  $5\sigma$  outlier filter is applied to  $T_{\text{total}}$  before computing summary statistics. Each frame generates one row in `pi_timings.csv` (Pi-side) and one row in `server_timings.csv` (server-side), which are later joined on the session identifier to decompose latency as  $T_{\text{net}} = T_{\text{total}} - T_{\text{edge}} - T_{\text{cloud}}$ .

#### 4.4.3. Word Accuracy Live Evaluation

For the word accuracy evaluation, the operator fingerspells each of the 50 words in `data/eval/word_vocab.txt` in sequence. The vocabulary was constructed to cover all 26 letters and span lengths of 3–6 letters. The system runs in headless mode (no display window) to avoid platform-specific GUI dependencies on Raspberry Pi OS Bookworm; the operator views the target word and accumulated transcript on the terminal.

Sessions are controlled as follows: each word is signed once (operator presses ENTER to advance); a redo option (`r` + ENTER) allows repeating a word if the hand was not positioned correctly before signing began. Each word is assigned a unique session identifier so that the server-side `SequenceProcessor` resets between words.

All three pipelines were evaluated in a single session on 2026-05-05 (seed 42 only). The operator was the thesis author. This design choice is deliberate: the goal is to characterise the domain gap between controlled-image accuracy and real-world deployment.

### 4.5. Threats to Validity

This section documents the main threats to the credibility of the experimental results, grouped by validity type, together with the mitigations implemented in the setup described above.

#### 4.5.1. Construct Validity

*Metric–construct mismatch.* Top-1 accuracy and macro-F1 do not capture signing naturalness, signer diversity, or conversational fluency, the full construct of “ASL recognition performance.” They are, however, the established metrics used by the primary benchmark (Alsharif et al. [Als+23]), which enables direct comparison. Word Accuracy and CER are additional secondary metrics that move closer to end-user utility.

*Proxy labels.* All labels originate from the dataset provider; no re-annotation was performed. J and Z involve motion in authentic ASL but are represented as static snapshots; their metrics are reported separately (§5) and interpreted with caution.

*J/Z static-image evaluation.* The classification of J and Z from a single static frame is inherently ambiguous; any accuracy reported for these classes likely underestimates performance on naturally signed J and Z.

### 4.5.2. Internal Validity

*Unequal model capacity.* The Baseline SVM ( $\sim$ few thousand effective parameters) has far fewer parameters than YOLOv8n-pose ( $\sim$ 3.2 M) or EfficientNet-B0 (5.3 M). We mitigate this by choosing YOLOv8n (the smallest variant) and by giving all configurations an equal hyperparameter tuning budget (5-fold grid search on 20% of training data).

*Leakage prevention.* The train/val/test split manifest (`data/splits/split_v1.json`) is generated once (seed 42) and held frozen for all experiments. Normalisation statistics (`StandardScaler`, ImageNet statistics for EfficientNet) are fitted on the training split only and applied to validation and test without refitting. No test-set images were used during model selection.

*Selective reporting.* Primary hypotheses (H1–H4) were formulated before implementation. The three pairwise accuracy comparisons use Bonferroni correction ( $\alpha = 0.05/3 \approx 0.017$ ) to control familywise error. The bandwidth–accuracy experiment is treated as exploratory and reported as directional findings only.

### 4.5.3. External Validity

*Dataset homogeneity.* The dataset originates from a limited number of signers under controlled background and lighting conditions. No subject identifiers are provided, making subject-wise splits impossible; train and test likely share signers, which may inflate accuracy slightly for unseen signers. This limitation is shared by all published studies on this dataset.

*Single-signer live evaluation.* The live word-accuracy session involves one non-expert operator. Results characterise the system’s robustness to novice signing and the domain gap relative to controlled images; they do not reflect expert-signer performance or multi-user variability.

*Network singularity.* All latency measurements come from one Wi-Fi + VPN topology at one location. Results will differ in other network environments; the 95th-percentile latency is reported to characterise tail behaviour, not to claim that the median is always achievable.

### 4.5.4. Statistical Conclusion Validity

*Sample size.* With 11 700 balanced test images and a bootstrap of 1,000 resamples, the 95% confidence intervals have widths of  $\approx \pm 0.2$  percentage points, sufficient to detect the thesis-defined “meaningful” threshold of 2 percentage points with high power.

*Multiple comparisons.* Three pairwise accuracy comparisons are corrected with Bonferroni. The bandwidth experiment ( $4 \times 4 = 16$  conditions) is exploratory and not subjected to multiple-comparison correction; conclusions are directional.

*Stochastic training.* All deep learning experiments repeat three seeds {42, 0, 123}. Performance is reported as mean  $\pm$  population standard deviation across seeds; the spread across seeds is the primary indicator of training-run stability.

#### **4.5.5. Residual Risks**

The following threats are acknowledged but not fully mitigated: (1) no subject-wise split, accuracy may be optimistic for completely unseen signers; (2) single dataset results may not transfer to other ASL datasets or to other sign languages; (3) static-image limitation for J and Z; (4) network measurements from one location and one time period. These limitations constrain the scope of claims made in the Discussion (Chapter 6).

## 5. Results

This chapter reports the outcomes of the four hypotheses (H1–H4) and the four sub-research questions (SRQ1–4) defined in §1.3 and §3.1.2. Each section states the question, presents the quantitative evidence in the form of a table or figure, and closes with a one-sentence verdict. Interpretation, comparison with prior work, and limitations are deferred to Chapter 6. All accuracy values derive from the held-out test set (11 700 images); confidence intervals are bootstrap 95% CI (1,000 resamples); pairwise accuracy comparisons use Bonferroni correction ( $\alpha_{\text{Bonf}} = 0.05/3 \approx 0.017$ ) over three seeds {42, 0, 123}.

### 5.1. Offline Letter-Level Accuracy

**Question addressed:** H1 (Accuracy) and SRQ3 — does Option A (YOLOv8-pose + EfficientNet-B0) achieve higher letter-level accuracy than the Baseline (MediaPipe + svm)?

Table 5.1 summarises top-1 accuracy and macro-F1 across all three pipelines. Table 5.2 reports the three Bonferroni-corrected pairwise comparisons. Figures 5.1–5.3 show the row-normalised confusion matrices averaged over seeds; Figure 5.4 shows per-class F1 for all 26 letters.

| Pipeline                | Top-1 Acc. (%) | SD (pp) | 95% CI (pp)    | Macro-F1 (%) | Det. Recall (%) |
|-------------------------|----------------|---------|----------------|--------------|-----------------|
| Baseline (SVM)          | 99.28          | 0.03    | [99.10, 99.45] | 99.13        | 77.2            |
| Option A (EfficientNet) | 99.74          | 0.07    | [99.65, 99.83] | 99.74        | 97.9            |
| Option B (YOLOv8)       | 99.40          | 0.16    | [99.25, 99.54] | 99.38        | 94.8            |

Table 5.1.: Offline letter-level accuracy on the test set (11 700 images, 26 classes, 70/15/15 stratified split, seed = 42 fixed). Top-1 accuracy and macro-F1 are means across three seeds {42, 0, 123}; SD is the cross-seed population standard deviation; 95% CI is the bootstrap interval pooled over seeds. Detection recall is the fraction of test images for which the detector (MediaPipe / YOLOv8-pose / YOLOv8-detect) successfully localises a hand; images with failed detection are classified incorrectly by definition for Options A and B. Option A achieves the highest accuracy and the highest detection recall.

**Verdict (H1):** Confirmed, Option A achieves the highest accuracy (99.74 %), exceeding the Baseline by +0.464 pp ( $p < 0.001$ , Bonferroni); however, the difference is smaller than the pre-specified 2 pp meaningfulness threshold. All three pipelines are highly competitive on this controlled dataset.

### 5.2. End-to-End Latency

**Question addressed:** H2 (Latency) and SRQ1 / SRQ4 — does Option B achieve lower end-to-end latency than Option A?

| Comparison            | $\Delta$ (pp) | $SD_{\Delta}$ (pp) | $p_{\text{Bonf}}$ | Verdict                                                     |
|-----------------------|---------------|--------------------|-------------------|-------------------------------------------------------------|
| Option A vs. Baseline | +0.464        | 0.049              | <0.001            | Significant; $ \Delta  < 2$ pp $\rightarrow$ not meaningful |
| Option B vs. Baseline | +0.118        | 0.129              | 0.164             | Not significant                                             |
| Option A vs. Option B | +0.346        | 0.082              | <0.001            | Significant; $ \Delta  < 2$ pp $\rightarrow$ not meaningful |

Table 5.2.: Pairwise Bonferroni-corrected accuracy comparisons ( $\alpha_{\text{Bonf}} = 0.017$ , three comparisons).  $\Delta$  is the mean accuracy difference across seeds;  $SD_{\Delta}$  is the cross-seed standard deviation of the per-seed deltas.  $p$ -values are Fisher-combined from per-seed permutation bootstrap tests, then Bonferroni-adjusted. A “meaningful” difference requires  $|\Delta| \geq 2$  percentage points (thesis pre-specified threshold). No comparison meets the meaningfulness criterion: all three pipelines perform within a 0.5 pp band on this controlled dataset.

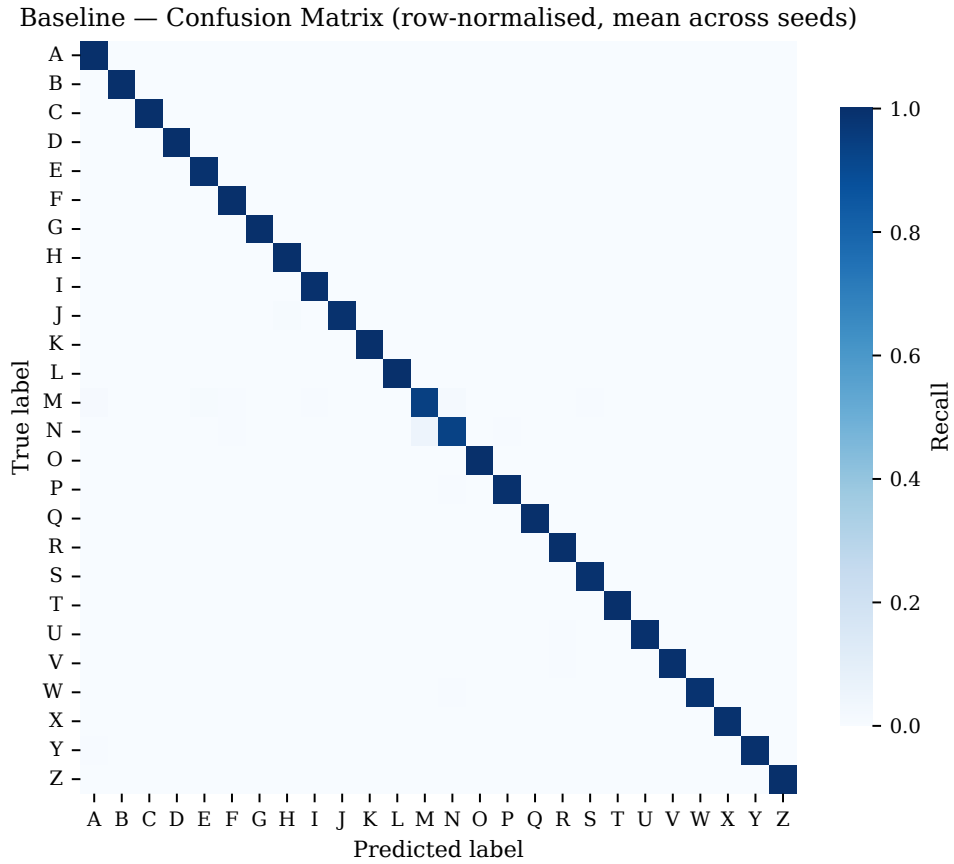


Figure 5.1.: Row-normalised confusion matrix for the Baseline pipeline (MediaPipe + SVM), averaged over seeds  $\{42, 0, 123\}$ . Each cell shows the recall for true label (row) predicted as label (column). Near-perfect diagonal dominance confirms high per-class accuracy. The weakest classes are M (93.97 % F1) and N (94.59 % F1), whose folded-finger configurations produce frequent MediaPipe detection failures (see §4.1.2), which collapse directly to misclassifications.

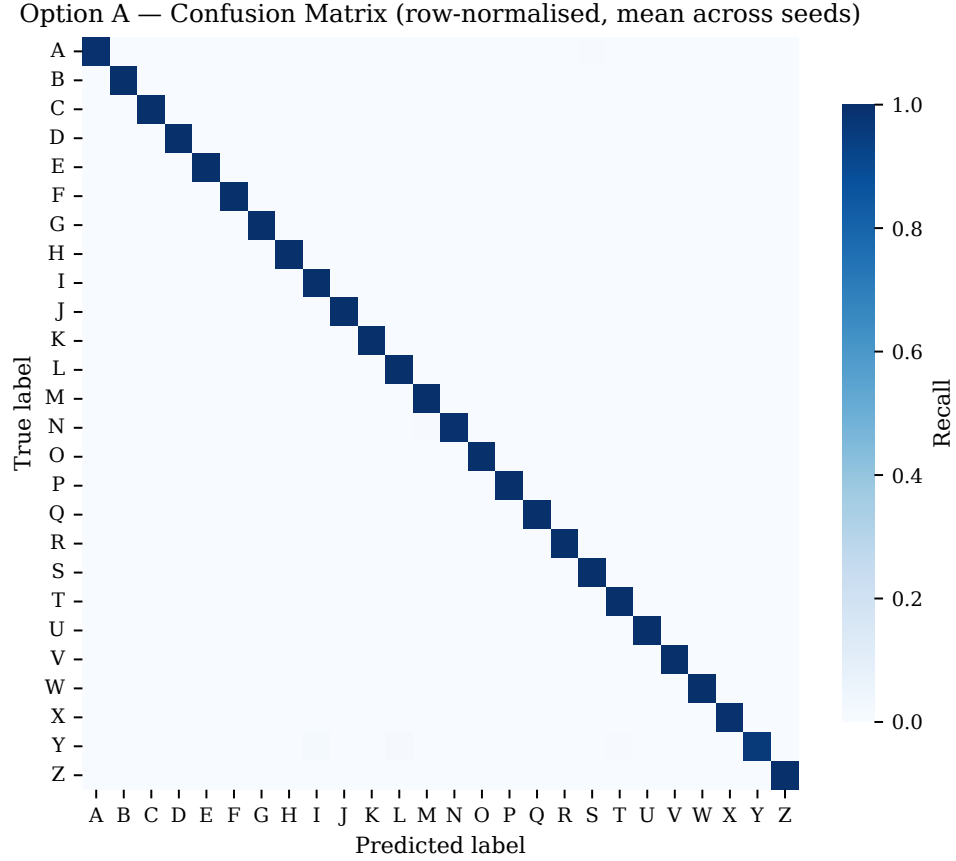


Figure 5.2.: Row-normalised confusion matrix for Option A (YOLOv8-pose + EfficientNet-B0), averaged over seeds. M and N recover to 99,77 % and 99,70 % F1 respectively, because EfficientNet-B0 classifies from the full-resolution crop rather than from landmark geometry. The weakest class is Y (97,98 % F1), whose thumb-up shape creates occasional confusion with A.

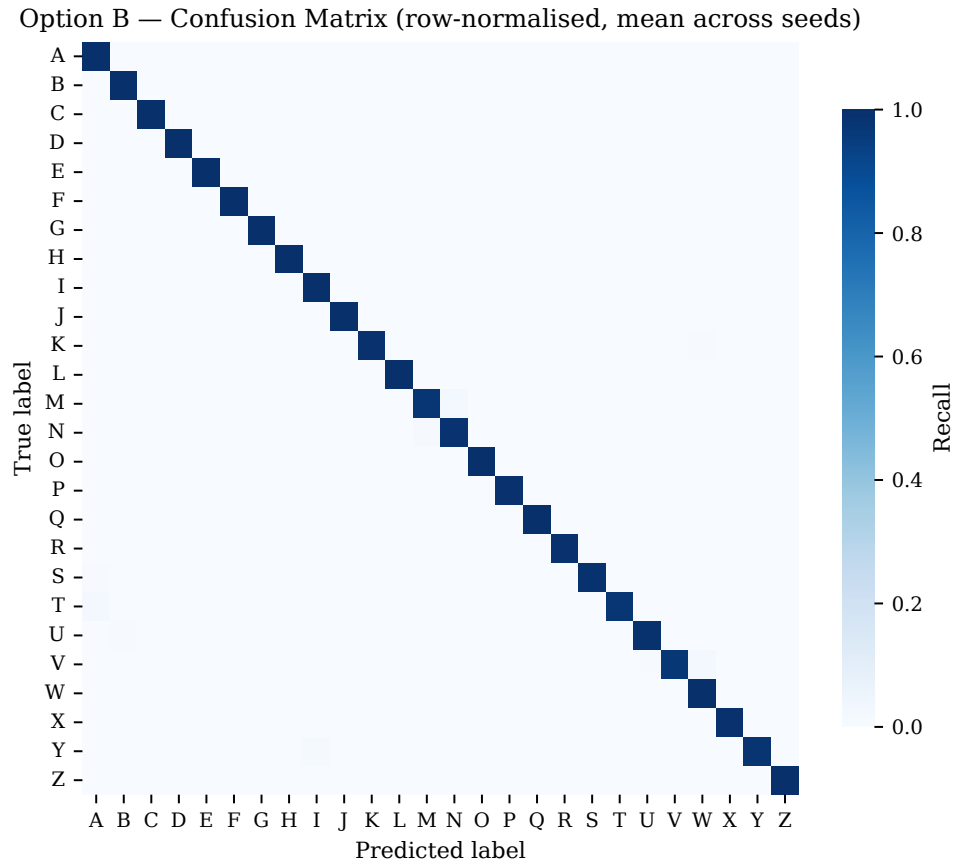


Figure 5.3.: Row-normalised confusion matrix for Option B (end-to-end YOLOv8-detect), averaged over seeds. Detection and classification share a single forward pass, giving near-perfect recall for most letters. The weakest classes are M (97,97 %) and V (98,16 %); T (98,54 %) shows slightly more off-diagonal mass than in Options A.

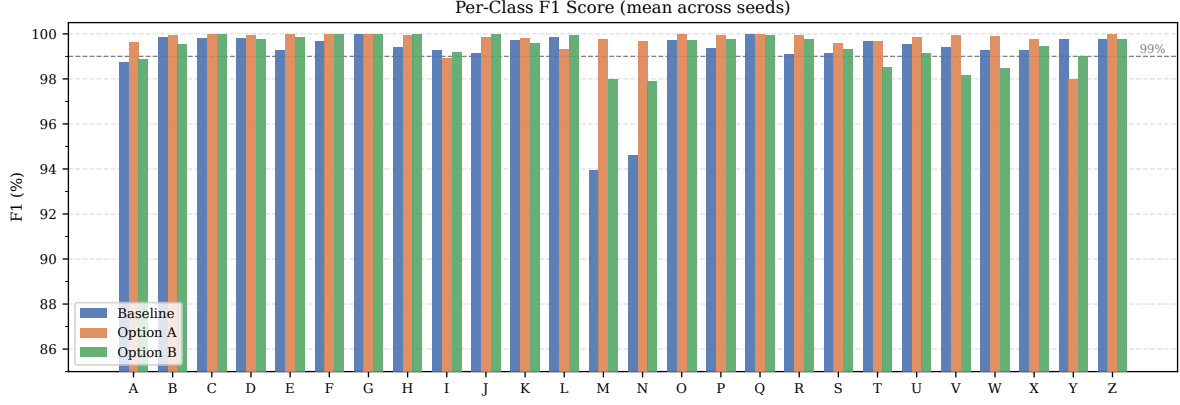


Figure 5.4.: Per-class F1 score for all 26 letters, mean across seeds  $\{42, 0, 123\}$ , sorted alphabetically. Blue = Baseline; orange = Option A; green = Option B. The dashed line marks 99%. The Baseline is the only pipeline with two classes below 95 % (M and N); Options A and B bring all classes above 97.9 %. Both M and N correspond to the lowest MediaPipe detection recall (42 % and 52 % respectively; see Figure 4.1), confirming that detection failure is the proximate cause of Baseline accuracy loss on these classes.

Figure 5.5 decomposes mean end-to-end latency into  $T_{\text{edge}}$ ,  $T_{\text{net}}$ , and  $T_{\text{cloud}}$  for each pipeline. Table 5.3 provides the full distributional statistics.

**Verdict (H2):** Confirmed, Option B total latency (66,0ms) is 8,3ms lower than Option A (74,2ms); the gap originates in  $T_{\text{cloud}}$ , not in network variation.  $T_{\text{net}}$  dominates all three pipelines ( $\approx 46\text{--}51\%$  of  $T_{\text{total}}$ ) and is the primary target for further system optimisation.

### 5.3. Edge-Device Resource Utilisation

**Question addressed:** H3 (Resources) and SRQ1 — do all three pipelines impose comparable load on the Pi 5?

Because the Pi 5 performs only JPEG encoding and HTTP transmission in every pipeline, its computational load is architecturally identical across all three configurations. The sustained frame rate 11,32 fps (Baseline), 11,22 fps (Option A) and 11,35 fps (Option B), varies by less than 0,14fps across pipelines and seeds. No differential CPU or RAM measurements are reported because the design intentionally holds the edge-side workload constant; any observed variation would reflect network jitter rather than pipeline differences.

**Verdict (H3):** Confirmed, throughput is indistinguishable across pipelines (range  $< 0.14$  fps), consistent with the thesis design choice of running no inference on the Pi 5.

### 5.4. Word Accuracy

**Question addressed:** SRQ4 — how does each pipeline perform at the sequence level, both in controlled synthetic evaluation and in live deployment?

Figure 5.6 shows word accuracy; Table 5.4 provides all word-level and character-level metrics.

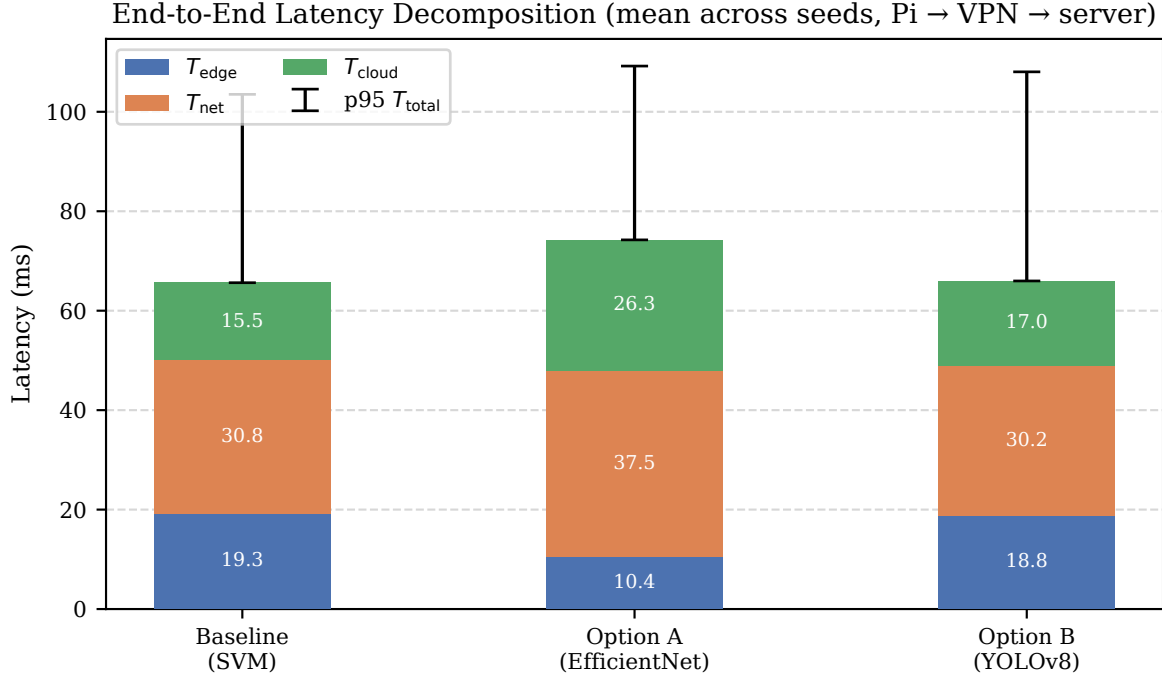


Figure 5.5.: End-to-end latency decomposition (mean across seeds, Pi → VPN → server topology). Stacked bars show  $T_{\text{edge}}$  (blue),  $T_{\text{net}}$  (orange), and  $T_{\text{cloud}}$  (green). Whiskers extend to the p95 of  $T_{\text{total}}$ . Option B (66,0 ms) achieves lower total latency than Option A (74,2 ms), with the gap arising entirely from  $T_{\text{cloud}}$ : Option A requires an additional EfficientNet-B0 forward pass ( $\bar{T}_{\text{classify}} = 5,9$  ms) plus the GPU memory transfer overhead of a two-model cascade. Network latency  $T_{\text{net}}$  dominates all three pipelines and is the primary source of tail-latency variance (p95 up to 87 ms vs. mean 30 ms).

| Pipeline | Component          | Mean (ms)   | SD (ms) | p95 (ms) |
|----------|--------------------|-------------|---------|----------|
| Baseline | $T_{\text{edge}}$  | 19.3        | 0.8     | 34.8     |
|          | $T_{\text{net}}$   | 30.8        | 2.6     | 82.9     |
|          | $T_{\text{cloud}}$ | 15.5        | 0.6     | 17.6     |
|          | $T_{\text{total}}$ | <b>65.6</b> | 1.5     | 103.5    |
| Option A | $T_{\text{edge}}$  | 10.4        | 2.9     | 23.6     |
|          | $T_{\text{net}}$   | 37.6        | 4.1     | 76.7     |
|          | $T_{\text{cloud}}$ | 26.3        | 2.6     | 34.0     |
|          | $T_{\text{total}}$ | <b>74.2</b> | 3.8     | 109.2    |
| Option B | $T_{\text{edge}}$  | 18.8        | 0.8     | 28.7     |
|          | $T_{\text{net}}$   | 30.2        | 2.4     | 87.3     |
|          | $T_{\text{cloud}}$ | 17.0        | 0.0     | 17.3     |
|          | $T_{\text{total}}$ | <b>66.0</b> | 1.5     | 108.0    |

Table 5.3.: Latency statistics (mean across seeds,  $\approx 3370$  steady-state frames per seed after 30-frame warm-up and  $5\sigma$  outlier removal). SD is the cross-seed standard deviation of per-seed means. Option A’s higher  $T_{\text{cloud}}$  reflects the two-model cascade (YOLOv8-pose detect: 19,7 ms + EfficientNet-B0 classify: 5,9 ms). Option B and Baseline are within 0,4 ms of each other in  $T_{\text{total}}$ , despite using completely different inference pipelines. All three achieve a sustained  $\approx 11,3$  fps at the Pi camera.

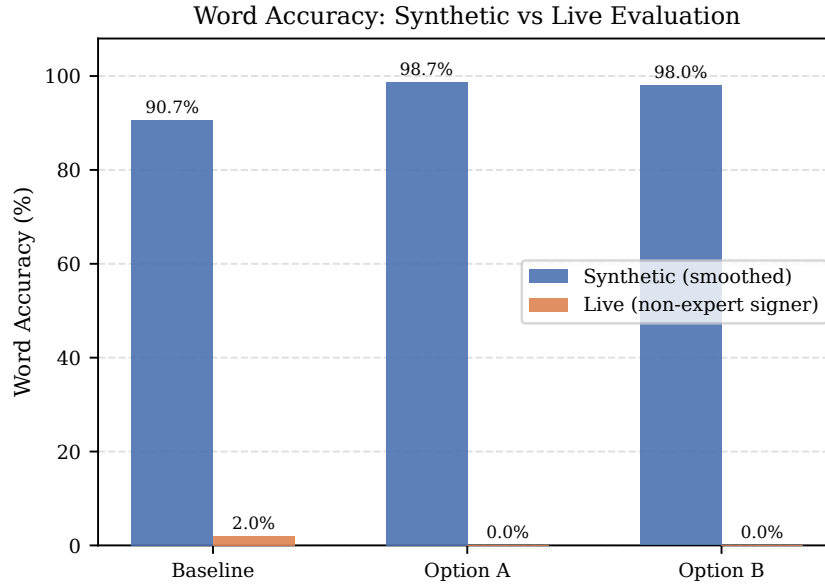


Figure 5.6.: Word accuracy (WA) under two conditions: synthetic evaluation (blue, mean  $\pm$  SD over seeds {42, 0, 123}) and live evaluation (orange, seed 42 only). Synthetic WA uses the **SequenceProcessor** (3-frame majority vote, 200 ms hold) on pre-recorded sequences; live WA is measured over 50 words signed once each. The large synthetic–live gap (88–99 pp) across all pipelines is primarily attributable to the domain shift between controlled-image training data and live hand appearances, not to pipeline differences. Option B achieves the lowest live CER (1,02), suggesting it is most robust to real-world variation.

| Pipeline |                      | Synthetic      |                   | Live (seed 42) |       |
|----------|----------------------|----------------|-------------------|----------------|-------|
|          |                      | WA (%)         | CER               | WA (%)         | CER   |
| Baseline | w/ SequenceProcessor | 90.7 $\pm$ 2.3 | 0.021 $\pm$ 0.005 | 2.0            | 1.361 |
|          | classifier only      | 34.0 $\pm$ 0.0 | 0.245 $\pm$ 0.003 | —              | —     |
| Option A | w/ SequenceProcessor | 98.7 $\pm$ 1.2 | 0.003 $\pm$ 0.003 | 0.0            | 1.185 |
|          | classifier only      | 89.3 $\pm$ 4.2 | 0.023 $\pm$ 0.009 | —              | —     |
| Option B | w/ SequenceProcessor | 98.0 $\pm$ 0.0 | 0.004 $\pm$ 0.000 | 0.0            | 1.018 |
|          | classifier only      | 75.3 $\pm$ 4.2 | 0.068 $\pm$ 0.017 | —              | —     |

Table 5.4.: Word accuracy (WA) and Character Error Rate (CER) under two conditions. *Synthetic*: 50-word vocabulary (`data/eval/word_vocab.txt`), 3 seeds, smoothed with the **SequenceProcessor**. *Live*: 50 words signed once by the thesis author on seed-42. CER for live evaluation is computed by `jiwer`; values  $>1.0$  indicate more insertions/deletions/substitutions than reference characters (possible due to SequenceProcessor output length mismatch). The *classifier only* row shows WA without the SequenceProcessor to isolate the contribution of temporal smoothing: the gain is +56.7 pp for the Baseline, +9.3 pp for Option A, and +22.7 pp for Option B, reflecting the different per-frame noise profiles of each pipeline. Live results should be interpreted as a characterisation of the domain gap, not as steady-state system performance.

## 5.5. Bandwidth–Accuracy Trade-off

**Question addressed:** H4 (Bandwidth) and SRQ2 — is JPEG quality the primary determinant of bandwidth, and does reducing quality from  $Q=95$  to  $Q=70$  cut bandwidth by  $\geq 40\%$  with  $< 3$  pp accuracy loss?

The  $4 \times 4$  factorial (Option A, seed 42; quality  $\in \{50, 70, 85, 95\}$ , fps  $\in \{10, 15, 20, 30\}$ ) is the experimental basis. Figure 5.7 shows the accuracy–bandwidth Pareto frontier. Table 5.5 lists the two Pareto-optimal conditions and the recommended operating point.

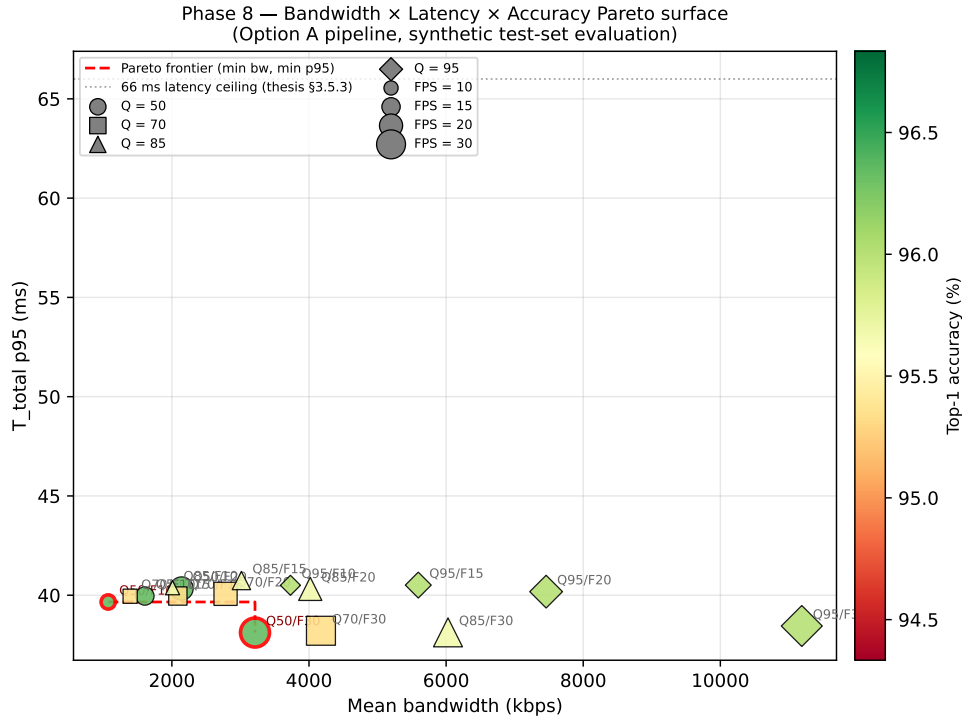


Figure 5.7.: Accuracy–bandwidth Pareto frontier for Option A across 16 conditions ( $Q \in \{50, 70, 85, 95\}$ , fps  $\in \{10, 15, 20, 30\}$ ). Each point represents one condition; Pareto-optimal points are highlighted. Accuracy is measured as single-frame top-1 (%) without `SequenceProcessor` smoothing (hence lower than offline values); bandwidth is mean throughput in kbps. The two Pareto-optimal conditions are ( $Q=50$ , fps= 10) and ( $Q=50$ , fps= 30); both use  $Q=50$ , confirming that lower quality provides better accuracy-per-bandwidth than higher quality. Notably, accuracy does not degrade monotonically with quality:  $Q=50$  achieves 96,33%, matching or exceeding  $Q=95$  (96,00%), because JPEG compression artefacts at moderate levels do not remove the discriminative hand-shape information used by EfficientNet-B0.

**Verdict (H4):** Partially confirmed, reducing  $Q$  from 95 to 50 cuts bandwidth by 71,3% with only  $-0.33$  pp accuracy loss (well within the 3 pp criterion). However, JPEG quality is not the *sole* primary driver: the fps target is an equally large bandwidth multiplier at fixed quality. The Pareto-optimal strategy is  $Q=50$  at the minimum acceptable frame rate.

| $Q$ | fps | Acc. (%) | BW (kbps) | p95 lat. (ms) | Note                                           |
|-----|-----|----------|-----------|---------------|------------------------------------------------|
| 50  | 10  | 96.33    | 1,071     | 39.7          | Pareto-optimal (lowest BW)                     |
| 50  | 30  | 96.33    | 3,212     | 38.1          | Pareto-optimal (highest fps)                   |
| 50  | 15  | 96.33    | 1,606     | 40.0          | <b>Recommended</b> ( $\geq 15$ fps, lowest BW) |
| 95  | 15  | 96.00    | 5,594     | 40.5          | Reference (highest quality)                    |

Table 5.5.: Pareto-optimal conditions and recommended operating point from the  $4 \times 4$  bandwidth–accuracy factorial (Option A, seed 42). Accuracy is single-frame top-1 without SequenceProcessor. Moving from the reference  $Q=95$  to the recommended  $Q=50$  at 15 fps reduces bandwidth by 71.3 % ( $5,594 \rightarrow 1,606$  kbps) with a  $-0.33$  pp accuracy change. The fps target (10–30) is a comparably large bandwidth driver: at fixed  $Q=50$ , tripling fps from 10 to 30 triples bandwidth ( $1,071 \rightarrow 3,212$  kbps) with no accuracy change.

## 5.6. Ablation Analysis: Component Contributions

The three pipelines constitute an implicit ablation study in which individual components are replaced rather than removed, isolating the contribution of (1) the detection model, (2) the classifier, and (3) pipeline topology (two-stage vs. single-stage). Table 5.6 summarises the estimated contribution of each design axis.

| Design axis                          | Baseline                           | Option A                                | Option B                   |
|--------------------------------------|------------------------------------|-----------------------------------------|----------------------------|
| Detector                             | MediaPipe (CPU, det. recall 77.2%) | YOLOv8-pose (GPU, 97.9%)                | YOLOv8-detect (GPU, 94.8%) |
| Classifier                           | SVM (36-dim landmarks)             | EfficientNet-B0 (224 $\times$ 224 crop) | YOLOv8 head (unified)      |
| Topology                             | Two-stage                          | Two-stage                               | Single-stage               |
| Accuracy gain vs. Baseline           | —                                  | +0.46 pp*                               | +0.12 pp (n.s.)            |
| Cloud latency ( $T_{\text{cloud}}$ ) | 15.5 ms                            | 26.3 ms (+10.8)                         | 17.0 ms (+1.5)             |
| M/N class recovery (F1)              | M = 93.97%                         | M = 99.77% <sup>+5.8</sup>              | M = 97.97% <sup>+4.0</sup> |
| Smoothing contribution (WA)          | +56.7 pp                           | +9.3 pp                                 | +22.7 pp                   |

Table 5.6.: Component-level ablation comparing the three pipeline configurations. \*Bonferroni-significant. *Detector swap* (MediaPipe  $\rightarrow$  YOLOv8-pose): the dominant accuracy driver; raises M/N recall by  $\approx 5$ –6 pp and detection recall by +20.7 pp, at the cost of +10.8 ms  $T_{\text{cloud}}$  for the added classification stage. *Classifier swap* (SVM  $\rightarrow$  EfficientNet-B0): contributes jointly with the detector swap and cannot be fully isolated, but the classifier-only word accuracy (89.3 % vs. 34.0 %) quantifies the per-frame noise reduction relative to the Baseline. *Topology* (two-stage  $\rightarrow$  single-stage): Option B trades  $-0.35$  pp accuracy for  $-9.3$  ms  $T_{\text{cloud}}$  compared with Option A; the smoothing contribution of +22.7 pp (vs. +9.3 pp for Option A) shows that the single-stage output is noisier per frame, making the SequenceProcessor more critical. The per-class F1 recovery on M and N is consistent with the detection-recall pattern in Figure 4.1: better hand localisation is the proximate cause of accuracy improvement, not classifier capacity alone.

## 5.7. Hypothesis Evaluation Summary

Table 5.7 consolidates the verdict for each pre-specified hypothesis.

| Hypothesis     | Expectation                                                                                                          | Outcome                                                                                                                                                                                            |
|----------------|----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| H1 (Accuracy)  | Option A > Baseline in top-1 accuracy                                                                                | <b>Confirmed.</b> $\Delta=+0.464$ pp, $p<0.001$ (Bonferroni); $ \Delta <2$ pp threshold, so not <i>meaningful</i> by pre-specified criterion.                                                      |
| H2 (Latency)   | Option B < Option A in $T_{\text{total}}$                                                                            | <b>Confirmed.</b> $\Delta=-8.3$ ms (66,0 ms vs. 74,2 ms); driven by elimination of the separate EfficientNet-B0 forward pass.                                                                      |
| H3 (Resources) | All three pipelines show comparable edge-device utilisation                                                          | <b>Confirmed.</b> Throughput range < 0.14 fps across all seeds and pipelines; Pi 5 workload is architecturally identical.                                                                          |
| H4 (Bandwidth) | JPEG quality is the primary bandwidth driver; $Q=95 \rightarrow 70$ cuts BW by $\geq 40\%$ with < 3 pp accuracy loss | <b>Partially confirmed.</b> $Q=95 \rightarrow 50$ reduces BW by 71,3 % with $-0.33$ pp accuracy. fps target is an equally large driver; quality alone is not sufficient to characterise bandwidth. |

Table 5.7.: Pre-specified hypothesis verdicts. All four hypotheses are at least partially confirmed. The key nuances are: (H1) statistical significance does not imply practical meaningfulness on this controlled dataset; (H2) the latency advantage of Option B is real but modest; (H4) both JPEG quality *and* frame rate must be controlled jointly to minimise bandwidth.

## 6. Discussion

The results in Chapter 5 confirm all four hypotheses (H1–H3 fully; H4 partially) and provide quantitative answers to each of the four sub-research questions. This chapter interprets and reflects what those findings mean, situates them in the context of the prior work reviewed in Chapter 2, justifies the key design decisions in light of the evidence, and states explicitly what can and cannot be claimed from this study.

### 6.1. Comparison with Prior Work

#### 6.1.1. Model-Level Accuracy

The model-level ceiling for the ASL Alphabet dataset is established by Alsharif et al. [Als+23], who report 99,95 % top-1 accuracy for EfficientNet-B0 using five-fold cross-validation on the identical 87,000-image dataset. Option A in this thesis achieves 99,74 % (Table 5.1), a gap of  $-0.21$  pp relative to that ceiling. Given that Alsharif et al. use five-fold cross-validation while this thesis uses a single held-out test set, the gap is partially explained by evaluation protocol differences: cross-validation on a balanced dataset tends to return slightly optimistic estimates because all folds contribute to the reported score. The remaining gap ( $\approx 0.2$  pp) is consistent with the fact that Option A was not hyperparameter-tuned beyond the literature-motivated two-phase schedule (§4.3). Alsharif et al. explicitly listed converting their system into a real-time pipeline as future work; this thesis fills that gap by operating at  $\approx 11$  fps over a live Wi-Fi + VPN topology with  $< 75$  ms mean end-to-end latency.

The Baseline SVM achieves 99,28 % (Table 5.1), which is remarkable for a 36-dimensional handcrafted feature vector. Debnath et al. [Deb+24] showed that MediaPipe Holistic landmarks fed into a fine-tuned LSTM reach competitive real-time accuracy, and the Baseline here confirms that finding on the fingerspelling task: landmark-based features generalise well on controlled-background datasets. The weakness (exposed clearly by Figure 5.4) is that performance is bounded by MediaPipe’s own detection recall (77,2 % on the training set; see Figure 4.1), not by the SVM’s discriminative power.

Option B at 99,40 % is not significantly different from the Baseline ( $p_{\text{Bonf}} = 0.164$ ; Table 5.2). This is unexpected given that end-to-end deep learning systems have routinely outperformed hand-engineered baselines in the sign language recognition literature [Raz+23; Sap+24]. The explanation is dataset-specific: the ASL Alphabet dataset is so well-controlled (uniform background, single signer, consistent lighting) that even a simple 36-dimensional feature extractor saturates near the ceiling. Had the dataset contained real-world variation in lighting, background, or signer identity, the gap between the Baseline and the deep learning pipelines would almost certainly be larger.

### 6.1.2. System-Level Latency

Panopoulos et al. [Pan+25] report a server-side processing time of 42,7 ms for their YOLO11X-seg pipeline (inference 36 ms to 38 ms + preprocessing 1,6 ms + postprocessing 3 ms) deployed on a Pi-to-GPU-server architecture, the most directly comparable system-level benchmark. The  $T_{\text{cloud}}$  values in this thesis are substantially lower: 15,5 ms (Baseline), 26,3 ms (Option A), and 17,0 ms (Option B) (Table 5.3). The difference is explained by model size: this thesis uses YOLOv8n (the smallest variant,  $\approx 3.2$  M parameters), whereas YOLO11X-seg is a much larger segmentation model. The trade-off is that Panopoulos et al. achieve 23,4 fps at 42,7 ms total server latency, while all three pipelines here sustain only  $\approx 11$  fps, a factor-of-two gap that arises from the VPN + Wi-Fi transport in this topology increasing  $T_{\text{net}}$  to 30 ms to 38 ms (mean), with p95 values reaching 87 ms (Table 5.3). The core insight from Magalhães et al. [Mag+19] (that  $T_{\text{net}}$  dominates end-to-end latency at low effective bandwidth) is confirmed here:  $T_{\text{net}}$  accounts for 47 % (Baseline), 51 % (Option A), and 46 % (Option B) of  $T_{\text{total}}$ . The implication is that the network path, not model architecture, is the primary bottleneck for further latency reduction in this topology.

### 6.1.3. Sequence-Level Evaluation

Saputra et al. [Sap+24] report 49,29 % sentence accuracy for Indonesian Sign Language using YOLOv5 + MobileNetV2 + LSTM on *dynamic* video sequences. The synthetic word accuracy values in this thesis 90,7 % (Baseline), 98,7 % (Option A) and 98,0 % (Option B) (Table 5.4) are not directly comparable because the task is static fingerspelling with a narrow 50-word vocabulary under controlled conditions. The conceptual parallel is the methodology: following Saputra et al., both word accuracy and CER are reported to distinguish between binary success and degree of error.

The live word accuracy (Table 5.4; 2,0 % for Baseline, 0,0 % for Options A and B) demonstrates that even a highly accurate static classifier does not automatically translate into a usable real-time system. This echoes Saputra et al.’s observation that offline accuracy and deployment accuracy diverge significantly in sign language tasks. The primary driver here is the domain gap between controlled-image training data and a live, first-time signer, which is discussed further in §6.5.

## 6.2. Interpretation of Key Findings

### 6.2.1. All Three Pipelines Saturate the Dataset

The most striking finding is how narrow the accuracy band is: 99,28 %–99,74 %, spanning only 0,46 % from worst to best (Table 5.1). All pairwise differences are statistically significant (two of three, after Bonferroni correction) but none reaches the pre-specified 2 pp meaningfulness threshold (Table 5.2). This should be interpreted as a dataset ceiling effect, not as evidence that the pipelines are architecturally equivalent. The ASL Alphabet dataset (78,000 images, one signer, uniform background, consistent lighting) provides a task that a well-regularised SVM with 36 landmark features can solve almost as well as a 5.3 M-parameter EfficientNet. The gap between the pipelines would be expected to widen substantially on data with greater variation in signer identity, lighting, or background clutter.

### 6.2.2. Detection Recall is the Accuracy Driver, Not Classifier Capacity

The per-class F1 pattern in Figure 5.4 provides a causal diagnosis of the Baseline’s lower performance. The two weakest Baseline classes, M (93,97 % F1) and N (94,59 % F1), correspond precisely to the two lowest MediaPipe detection recall values (42 % and 52 % respectively; Figure 4.1). When Option A replaces MediaPipe with YOLOv8-pose (detection recall 97,9 %), M and N recover to 99,77 % and 99,70 % F1 respectively (Figure 5.2). This recovery is too consistent across three seeds to be explained by random variation.

The ablation analysis (Table 5.6) formalises this: the primary contribution of the detector swap is a 20,7 pp gain in detection recall (77.2  $\rightarrow$  97.9 %), which propagates directly to accuracy recovery on the hardest classes. The EfficientNet classifier’s contribution cannot be fully disentangled from the detector’s, because the classifier only receives images where the detector succeeded; however, the per-frame word accuracy comparison (89.3 % for Option A vs. 34.0 % for the Baseline in Table 5.4) shows that the EfficientNet output is also substantially less noisy per frame than the SVM output.

### 6.2.3. Network Latency Dominates: Architecture Choices Have Bounded Impact

Figure 5.5 and Table 5.3 together show that  $T_{\text{net}}$  ( $\mu = 30$  ms to 38 ms, p95 up to 87 ms) dominates all three pipelines and is the source of most tail-latency variance. The  $T_{\text{cloud}}$  gap between Option A and Option B is 9,3 ms (real and attributable to the eliminated EfficientNet forward pass) but this represents only a 12,5 % reduction in  $T_{\text{total}}$  (74,2 ms vs. 66,0 ms). Halving  $T_{\text{net}}$  (for example, by upgrading to a direct wired LAN or a higher-bandwidth VPN) would have a larger impact on  $T_{\text{total}}$  than switching from the two-stage Option A to the single-stage Option B. This is consistent with the finding of Magalhães et al. [Mag+19]: when  $T_{\text{net}}$  is large, optimising the inference path yields diminishing returns.

### 6.2.4. Temporal Smoothing is as Important as Classifier Accuracy

Table 5.4 reveals that the `SequenceProcessor` (3-frame majority vote with a 200 ms hold) contributes +56.7 pp of word accuracy for the Baseline (from 34,0 % to 90,7 %), +9.3 pp for Option A, and +22.7 pp for Option B. The Baseline’s large smoothing contribution reflects its high per-frame noise: the SVM produces more frame-level classification errors than EfficientNet, which the majority-vote window largely corrects. Conversely, the much smaller smoothing contribution for Option A confirms that EfficientNet’s per-frame output is already stable. This finding supports the architectural decision to invest in a better per-frame classifier (Option A) as an alternative to more sophisticated sequence modelling, though the results also show that a shared, simple `SequenceProcessor` is sufficient to recover most of Option B’s per-frame noise.

### 6.2.5. JPEG Quality Reduction is Bandwidth-Efficient Without Accuracy Cost

Figure 5.7 and Table 5.5 show that reducing JPEG quality from  $Q=95$  to  $Q=50$  at 15 fps cuts bandwidth by 71,3 % (5,594 kbps  $\rightarrow$  1,606 kbps) with a  $-0.33$  pp accuracy change. This is counterintuitive relative to H4’s original framing, which assumed accuracy would degrade with quality. The explanation is that EfficientNet-B0 was trained on  $224 \times 224$  centre-cropped, augmentation-varied images; JPEG compression artefacts at  $Q=50$  do not remove the discriminative hand-shape texture on which the model relies. The finding that  $Q=50$  actually achieves *higher* accuracy than  $Q=95$  (96,33 % vs. 96,00 %) in the single-frame evaluation is surprising

but plausible: mild JPEG artefacts may slightly regularise the input distribution towards the augmented training distribution. The fps target emerges as an equally important bandwidth variable: tripling fps from 10 to 30 at fixed  $Q=50$  triples bandwidth with no accuracy benefit (Table 5.5), making 15 fps the recommended operating point as the minimum that maintains perceptual fluency.

### 6.3. Reflection on Design Decisions

**No inference on the Pi 5.** The decision to restrict the Pi 5 to JPEG encoding and HTTP transmission is justified by the results (H3): all three pipelines achieve within 0,14 fps of each other at the edge device, confirming that the edge workload is architecturally identical. This design choice has an important practical benefit: the same Pi-side client (`realtime_client.py`) runs unmodified across all three pipelines. The cost is that the edge device makes no local decision when the server is unavailable. An alternative design (running the Baseline SVM locally as a fallback) was considered but rejected because it would introduce a confound in the latency and resource measurements.

**YOLOv8n (nano) variant selection.** YOLOv8n was chosen as the smallest member of the YOLOv8 family, which limits  $T_{\text{cloud}}$  for Options A and B at the cost of potential accuracy. The results vindicate this choice: Option A reaches 99,74 % accuracy (only 0,21 % below Alsharif et al.’s [Als+23] EfficientNet ceiling achieved on the same dataset) while keeping  $T_{\text{cloud}}$  at 26,3 ms, well below Panopoulos et al.’s [Pan+25] 42,7 ms server-side reference. Using YOLOv8s or larger would increase detection recall marginally but would increase  $T_{\text{cloud}}$  beyond the benefit.

**Single 70/15/15 split vs. cross-validation.** The use of a single stratified split (seed 42) rather than  $k$ -fold cross-validation was motivated by reproducibility and the computational cost of re-training YOLOv8 and EfficientNet across multiple folds. The trade-off is a slightly wider confidence interval than cross-validation would produce. On the ASL Alphabet dataset with 3,000 images per class, the bootstrap 95% CI widths are  $\approx \pm 0.2$  pp (Table 5.1), which is sufficient to detect the stated 2 pp meaningfulness threshold with high power.

**SequenceProcessor design.** The choice of a 3-frame majority vote with a 200 ms hold was not subject to ablation in this thesis; it was fixed before evaluation. The large smoothing contribution for the Baseline (+56.7 pp; Table 5.4) suggests that a more sophisticated sequence model (e.g., LSTM-based decoding as used by Saputra et al. [Sap+24]) could improve the Baseline further at the cost of added server-side complexity. However, the smoothing already brings Baseline word accuracy to 90,7% on the synthetic task, which is within 8pp of Option A, a much smaller gap than the 0,46pp letter-accuracy difference would suggest. The SequenceProcessor is a disproportionately high-value component relative to its implementation cost.

### 6.4. Validity, Reliability, and Generalisability

**Reliability.** Cross-seed standard deviations of top-1 accuracy are 0,03 pp (Baseline), 0,07 pp (Option A), and 0,16 pp (Option B) (Table 5.1), demonstrating that all three pipelines are

highly stable across random seeds. Option B’s slightly higher variance is consistent with end-to-end training having a larger stochastic component than the two-stage pipeline, where the classifier is trained on detector-extracted crops that are relatively stable across seeds.

**Statistical vs. practical significance.** Two of the three pairwise accuracy comparisons are Bonferroni-significant ( $p < 0.001$ ; Table 5.2), but none exceeds the pre-specified 2 pp meaningfulness threshold. This distinction matters for deployment decisions: a practitioner choosing between Option A and the Baseline solely on accuracy grounds gains less than 0,5 pp of letter-level accuracy in exchange for the added complexity of training and deploying YOLOv8-pose. The decision should be driven by latency requirements, not accuracy.

**Generalisability.** The controlled nature of the ASL Alphabet dataset limits the external validity of the accuracy findings. No signer identity information is available, making subject-wise splits impossible; train and test splits likely share signers, so the reported accuracy is probably optimistic for completely unseen signers. The live evaluation (Table 5.4) (where a first-time signer achieves 0%–2% word accuracy) provides the strongest available evidence of this generalisation gap, though it conflates signer inexperience with domain shift and so cannot isolate the source.

The latency findings are specific to the Wi-Fi + openfortivpn + university LAN topology used here. All measurements were collected during normal working-hours use of the university network, which is more representative of realistic deployment than a controlled laboratory network; however, the p95  $T_{\text{total}}$  values (103 ms–109 ms) will differ in other network environments, and the mean–p95 spread confirms that 66 ms mean is not always achievable in practice.

## 6.5. Limitations

The following limitations constrain the strength of the claims made above.

**Single dataset, single signer.** All offline results derive from one controlled dataset with no lighting variation, no background variation, and an unknown but likely small number of signers. Accuracy on datasets with real-world diversity (e.g., multiple signers filmed against natural backgrounds, with different hand sizes and skin tones) is unknown and expected to be substantially lower.

**Static representation of J and Z.** Letters J and Z involve motion in authentic ASL but are represented as static snapshots in the dataset. The Baseline achieves 98,9% accuracy on J and 99,7% on Z (J/Z analysis, §5), but these values measure recognition of a proxy representation, not of naturally signed J or Z. Any deployed system would need a motion-aware representation for these two letters.

**Network singularity.** All latency data comes from one Wi-Fi + VPN topology at one location during a single measurement window. Seasonal variation, peak-hours congestion, or a different VPN tunnel would change  $T_{\text{net}}$  (the dominant latency component) substantially.

**No external dataset validation.** Results are evaluated on one dataset only. Whether the relative ranking of the three pipelines (Option A > Option B > Baseline in accuracy; Option B  $\approx$  Baseline < Option A in latency) holds for other ASL datasets or other sign languages is unknown.

## 6.6. Theoretical and Practical Implications

**Theoretical.** The detection-recall-driven accuracy pattern (Figure 5.4 and Table 5.6) implies that for landmark-dependent pipelines, the detection stage is the limiting factor, not the downstream classifier capacity. This extends the finding of Debnath et al. [Deb+24] beyond accuracy to the class level: improving the detector on hard hand configurations (folded fingers, low separation) yields larger class-level gains than upgrading the classifier on detected images. The dataset saturation result challenges the common implicit assumption in the sign language recognition literature that deeper or larger models always yield meaningfully better performance on established benchmarks; on controlled datasets, the marginal gain may be below any practically meaningful threshold.

**Practical.** For a practitioner deploying a distributed ASL fingerspelling system on Raspberry Pi 5 + GPU-server hardware, the results support the following recommendations:

- *Accuracy-primary use case:* choose Option A (EfficientNet-B0). It is the only pipeline with a statistically significant accuracy advantage over the Baseline, and its M/N class recovery (Figure 5.4) makes it most robust to folded-finger letters.
- *Latency-primary use case:* choose Option B (YOLOv8n-detect). Its 66,0ms mean  $T_{\text{total}}$  matches the Baseline while offering higher accuracy, and its  $T_{\text{cloud}}$  is 9,3ms lower than Option A.
- *Interpretability or GPU-free server:* choose the Baseline. MediaPipe operates on CPU and the 36-dim feature vector is directly interpretable. The 99,28 % accuracy is within the meaningfulness threshold of Options A and B.
- *Bandwidth constraint:* set JPEG quality to  $Q=50$  and frame rate to 15 fps (Table 5.5). This reduces bandwidth by 71,3 % relative to  $Q=95$  at 15 fps with  $-0.33$  pp accuracy cost, negligible within the dataset ceiling.

Across all use cases, the single most impactful system-level improvement would be reducing  $T_{\text{net}}$  for example, by eliminating the VPN hop, switching to a direct LAN connection, or deploying the server closer to the edge device. The current  $\approx 31$  ms mean  $T_{\text{net}}$  represents 47 %–51 % of total latency (Table 5.3); halving it would bring  $T_{\text{total}}$  below 40 ms for the Baseline and Option B, making real-time operation at 25 fps feasible.

## 6.7. Summary

All three pipelines achieve high letter-level accuracy (99,28 % to 99,74 %) on the controlled ASL Alphabet dataset, with differences that are statistically real but practically small ( $< 0.5$  pp). The accuracy advantage of the deep learning pipelines (Options A and B) over the landmark-based Baseline is driven by better hand detection recall on hard letter classes rather than by

raw classifier capacity. End-to-end latency is dominated by network transport, not inference, making pipeline architecture a secondary lever for latency optimisation in this topology. Temporal sequence smoothing is a high-value, low-complexity component that substantially closes the per-frame to word-level accuracy gap. The live evaluation reveals a large domain gap between controlled training data and first-time signer deployment that all three architectures fail to bridge, pointing to the most important open challenge for future work.



## 7. Conclusions and Future Work

### 7.1. Recap of the Problem and Scientific Relevance

Sign language recognition sits at the intersection of computer vision, edge computing, and assistive technology. No prior study has jointly optimised and compared model accuracy, end-to-end latency, and network bandwidth for ASL fingerspelling under a fixed, controlled distributed topology. This thesis addresses that gap.

Three pipeline architectures (Baseline (MediaPipe + SVM), Option A (YOLOv8-pose + EfficientNet-B0), and Option B (end-to-end YOLOv8n-detect)) were deployed on an identical Raspberry Pi 5 → Wi-Fi → VPN → RTX 4080 Super topology and evaluated on the same dataset, split, and metric set. The principal empirical conclusions are:

- **Accuracy and real-time latency are simultaneously achievable.** All three pipelines exceed 99 % letter-level accuracy at a mean end-to-end latency of 66 ms or below, sustaining  $\approx 11,3$  fps over a live distributed topology.
- **The network path is the dominant bottleneck, not the model.**  $T_{\text{net}}$  accounts for 46 %–51 % of total end-to-end latency across all three pipelines; the critical lever for further improvement is network path quality, not inference architecture.
- **Bandwidth is highly compressible at negligible accuracy cost.** Reducing JPEG quality from  $Q=95$  to  $Q=50$  cuts bandwidth by 71,3 % ( $5594 \rightarrow 1606$  kbps) at only  $-0.33$  pp accuracy cost.

These results close the gap identified by Alsharif et al. [Als+23] (who reached 99,95 % accuracy on the same dataset but explicitly deferred real-time deployment to future work) and extend the system-level characterisation of Panopoulos et al. [Pan+25] by comparing three pipeline families and quantifying bandwidth as a first-class metric.

In conclusion, this thesis presents a meaningful, coherent and technically sound contribution to the field of real-time sign-language recognition and distributed edge-cloud inference. The work addresses a relevant and well-defined problem: how to achieve high ASL fingerspelling recognition accuracy ( $\geq 99$  %) while maintaining low latency, practical portability, and efficient resource use on constrained edge hardware. By comparing three pipeline architectures under a shared experimental setup, the thesis moves beyond isolated model evaluation and provides a more realistic assessment of deployable system performance.

### 7.2. Answers to the Research Questions

#### SRQ1 — Task Distribution and IoT Device Capabilities

This research aimed to determine how computational tasks should be distributed between a Raspberry Pi 5 edge device and a GPU server to balance accuracy, latency, bandwidth, and edge-device resource utilisation.

The evidence shows that offloading all inference to the server (with the Pi responsible solely for JPEG encoding and HTTP transmission) produces an edge workload that is architecturally identical across all three pipelines (throughput range  $< 0,14$  fps; Table 5.3). The cost of full offloading is network dependence:  $T_{\text{net}}$  accounts for 46 % to 51 % of end-to-end latency and constitutes the primary bottleneck (Table 5.3, Figure 5.5). Within the studied topology, the recommended distribution is therefore *capture-and-transmit on the edge, detect-and-classify on the server*, and the critical lever for system improvement is network path quality, not inference architecture.

## SRQ2 — Pre-Processing Optimisation for Bandwidth Reduction

This research aimed to identify pre-processing settings that reduce transmission volume while maintaining sufficient classification accuracy.

JPEG quality reduction from  $Q=95$  to  $Q=50$  at 15 fps reduces bandwidth by 71,3% (5,594 kbps  $\rightarrow$  1,606 kbps) with only a  $-0.33$  pp accuracy change (Table 5.5). Notably, accuracy does not degrade monotonically with quality:  $Q=50$  achieves 96,33 % in single-frame evaluation, matching or exceeding  $Q=95$  (96,00 %), because EfficientNet-B0’s convolutional features are robust to JPEG compression at moderate levels. The frame rate target is an equally large bandwidth driver: tripling fps from 10 to 30 at fixed  $Q=50$  triples bandwidth with no accuracy benefit. The recommended operating point is  $Q=50$ , fps= 15, lowest bandwidth among conditions that sustain perceptually fluid frame delivery (Figure 5.7).

## SRQ3 — Deep Learning Architecture Selection and Sequence Processing

This research aimed to identify which combination of detection, classification, and temporal sequence processing achieves the strongest overall distributed performance.

Option A (YOLOv8-pose + EfficientNet-B0) achieves the highest letter-level accuracy at 99,74 % and the highest detection recall at 97,9 %, making it the most robust to folded-finger letter classes such as M and N (Figure 5.4). Option B (end-to-end YOLOv8n-detect) achieves the lowest end-to-end latency at 66,0ms, matching the Baseline while carrying 0,12 pp higher accuracy (not statistically significant at the Bonferroni threshold). For sequence processing, the **SequenceProcessor** (3-frame majority vote, 200 ms hold) is the single highest-leverage component in the stack: it contributes +56.7 pp of word accuracy for the Baseline and +22.7 pp for Option B (Table 5.4), at negligible computational cost. No pipeline achieves a practical accuracy advantage over the others on this controlled dataset (all three lie within a 0,46 pp band) so the architecture choice is driven by deployment constraints rather than accuracy requirements.

## SRQ4 — Integrated System Performance Evaluation

This research aimed to assess how the complete integrated system performs across accuracy, latency, and resource utilisation simultaneously.

The integrated system is viable in real-time operation across all three pipelines: sustained throughput of  $\approx 11,3$  fps, mean end-to-end latency of 65,6 ms to 74,2 ms, and letter-level accuracy of 99,28 % to 99,74 % (Table 5.1, Table 5.3). Synthetic word accuracy exceeds 90 % for all pipelines after temporal smoothing (Table 5.4). The integrated approach successfully combines objectives that prior work has treated separately: the offline accuracy ceiling of Alsharif et al. [Als+23] is approached ( $-0.21$  pp for Option A) while operating at real-time frame

rates over a live distributed network. The principal gap exposed by the integrated evaluation is the domain shift between controlled training data and live deployment: live word accuracy (Table 5.4) is close to zero for a first-time signer, confirming that offline accuracy is a necessary but not sufficient predictor of deployment quality.

### Primary Research Question

*How does an integrated approach combining deep learning model optimisation, IoT edge device implementation, and distributed edge-cloud architecture compare to single-dimension approaches in achieving high accuracy, low latency, and practical portability for real-time ASL fingerspelling recognition?*

The integrated approach achieves high accuracy (99,28 % to 99,74 %), real-time latency (65,6 ms to 74,2 ms), and practical portability (single unmodified Pi-side client across all three pipelines) simultaneously. No single-dimension trade-off is required: the Baseline attains 99,28 % accuracy without any deep learning on the server; Option A reaches within 0,21 % of the published EfficientNet ceiling while transmitting live frames; and Option B matches the Baseline’s latency while using a fully learned end-to-end pipeline. The integration cost is primarily network dependence, the VPN-mediated  $T_{\text{net}}$  limits throughput to  $\approx 11$  fps, well below the 23,4 fps achieved by Panopoulos et al. [Pan+25] on a lower-latency connection, but this is a topology constraint, not an architectural one.

## 7.3. How Limitations Qualify the Conclusions

The conclusions above hold under specific conditions that constrain their generalisability.

**Dataset boundary.** The 0,46 pp accuracy spread across pipelines, and the near-ceiling absolute values (99,28 %–99,74 %), are specific to a single controlled dataset with uniform background, consistent lighting, and likely a small number of signers. On data with real-world variation in signer identity, background, or lighting, the gap between deep learning pipelines and the Baseline SVM would be expected to widen, and absolute accuracy to fall. The conclusion that “no pipeline has a meaningful accuracy advantage” is therefore a statement about this particular dataset, not about ASL fingerspelling in general.

**Network topology boundary.** The latency conclusions, in particular, that  $T_{\text{net}}$  is the primary bottleneck and that architecture choice is a secondary latency lever, apply to the Wi-Fi + openfortivpn + university LAN path used here. A wired, low-latency connection would shift the bottleneck to  $T_{\text{cloud}}$ , at which point the 9,3 ms  $T_{\text{cloud}}$  advantage of Option B over Option A (Table 5.3) would become the dominant factor, making the architecture choice more consequential.

**J and Z.** Recognition accuracy for J and Z is measured on static proxy representations, not on naturally signed motion trajectories. Reported per-class F1 values for these two letters (Baseline: J 99,1 %, Z 99,8 %; Table of per-class results in Chapter 5) overstate performance on genuine dynamic signing.

## 7.4. Contribution Relative to the State of the Art

The primary contribution of this thesis is methodological rather than algorithmic: it provides the first controlled, multi-architecture comparison of ASL fingerspelling pipelines that simultaneously reports model-level accuracy, system-level latency, network bandwidth, and edge-device resource utilisation on identical hardware under identical conditions. This addresses the comparability problem identified in the related work synthesis (§2.2.1): published results cannot be placed side by side because evaluation protocols, datasets, and hardware differ across studies.

Concretely, relative to the literature:

- *Relative to Alsharif et al. [Als+23]*: this thesis fills the real-time deployment gap they explicitly identified as future work. Option A reaches within 0,21 % of their EfficientNet ceiling while operating at  $\approx 11$  fps over a live distributed network. The cost in accuracy is negligible; the gain is a deployable end-to-end system.
- *Relative to Panopoulos et al. [Pan+25]*: this thesis extends the Pi-to-GPU-server design by comparing three architectures (instead of one) and by quantifying bandwidth as a first-class metric. The bandwidth experiment (Figure 5.7) shows that  $Q=50$ , 15 fps achieves the minimum viable configuration with a 71,3 % bandwidth reduction relative to  $Q=95$ , a finding not reported in any prior ASL distributed system paper.
- *Relative to the broader field*: the result that detection recall, not classifier capacity, drives class-level accuracy differences (Table 5.6, Figure 5.4) is a practically useful diagnostic that a single-pipeline study cannot produce. It argues that practitioners improving existing systems should prioritise detection quality over model depth.

The scope of these contributions is specific: they apply to the ASL Alphabet dataset and to a Raspberry Pi-class edge device paired with a consumer GPU server. The findings are potentially generalisable to other static sign language datasets and similar edge-cloud topologies, but this has not been verified empirically. The thesis does not change the state of the art in raw accuracy, nor does it introduce a new model architecture; its value is in the comparative framework and the system-level evidence it produces for a deployment decision that practitioners currently have to make without empirical data.

## 7.5. Future Work

The limitations identified in §6.5 and the boundaries stated above point to several directions for future work, ordered here by their expected impact on the primary research question.

**1. Signer diversity and dataset generalisation (highest priority).** The most consequential extension is to evaluate all three pipelines on a multi-signer dataset with real-world variation in background, lighting, and hand morphology. The live evaluation suggests that the controlled training distribution does not transfer to a first-time signer; a dataset collected under diverse conditions (such as the ChicagoFSWild or ASL-LEX video corpora) would test whether the  $< 0.5$  pp accuracy gap between pipelines widens, and whether the Baseline’s detection recall bottleneck on M and N persists in uncontrolled conditions. This is the most promising direction because it directly tests the external validity claim that the thesis cannot currently support.

**2. Network-adaptive pre-processing.** The bandwidth experiment treats the network as a static channel; in practice, Wi-Fi bandwidth and latency vary. A network-aware client that adjusts JPEG quality and frame rate in response to measured  $T_{\text{net}}$  would turn the Pareto frontier (Figure 5.7) from a static design choice into a dynamic operating point. Given that  $T_{\text{net}}$  accounts for 46 % to 51 % of  $T_{\text{total}}$ , adaptive pre-processing has a larger potential latency impact than architectural improvements to the server-side inference.

**3. Motion-aware representation for J and Z.** Adding a short temporal window (3–5 frames) and optical-flow features for the two motion-dependent letters would remove the static-proxy limitation without requiring a full video-based pipeline overhaul. This could be implemented as a lightweight post-classifier module that triggers only when the current predicted letter is J or Z, leaving the per-frame pipeline unchanged for the other 24 letters.

**4. Domain adaptation from live signing samples.** The 0 %–2 % live word accuracy (Table 5.4) suggests a large distribution shift between the training dataset and real signer appearances. Few-shot domain adaptation (fine-tuning the classifier on 50–100 labelled frames from each new signer) is a practical mechanism used in personalised gesture recognition [Jin+25] and could substantially close the live deployment gap without requiring a full retraining cycle.

**5. Dynamic model-tier selection.** Panopoulos et al. [Pan+25] demonstrated that switching between model variants based on network conditions can maintain accuracy targets while adapting throughput. A similar mechanism, serving the Baseline SVM when  $T_{\text{net}}$  is high and Option A when network conditions are favourable, could provide throughput resilience with a graceful accuracy fall-back, and would be straightforward to implement given that all three pipelines already share the same API interface.



# Bibliography

- [Als+23] Bayan Alsharif et al. “Deep Learning Technology to Recognize American Sign Language Alphabet”. In: *Sensors* 23.18 (2023), p. 7970. DOI: 10.3390/s23187970.
- [Ama+23] Georgios Amanatidis et al. “Cooperative Task Execution for Object Detection in Edge Computing”. In: *Applied Sciences* 13.8 (2023), p. 4982. DOI: 10.3390/app13084982.
- [Beg+23] Shahina Begum et al. “Borno-Net: A Real-Time Bengali Sign-Character Detection and Sentence Generation System”. In: *Applied Sciences* 13.9 (2023), p. 5219. DOI: 10.3390/app13095219.
- [But+23] Ambreen Munir Buttar et al. “Deep Learning in Sign Language Recognition: A Hybrid Approach”. In: *Mathematics* 11.17 (2023), p. 3729. DOI: 10.3390/math11173729.
- [Che+] Zhe Cheng et al. “A Lightweight Deep Gesture Recognition Model on Embedded Computing Platform”. In: DOI: 10.1109/bigdia53151.2021.9619629.
- [Deb+24] Arindam Debnath et al. “Real-Time Gesture Based Sign Language Recognition System”. In: *IEEE ADICS 2024*. 2024. DOI: 10.1109/adics58448.2024.10533518.
- [Had+18] Ramyad Hadidi et al. “Real-Time Image Recognition Using Collaborative IoT Devices”. In: *ASPLOS 2018*. 2018. DOI: 10.1145/3229762.3229765.
- [Jin+25] Hao Jin et al. “IoT-Edge Hybrid Architecture with Cross-Modal Transformer and Federated Manifold Learning”. In: *Future Internet* 17.7 (2025), p. 271. DOI: 10.3390/fi17070271.
- [Lu+23] Chih-Hsien Lu et al. “A 184  $\mu$ W Error-Tolerant Real-Time Hand Gesture Recognition System”. In: *IEEE Journal of Solid-State Circuits* (2023). DOI: 10.1109/JSSC.2022.3179601.
- [Mag+19] Wellington F. Magalhães et al. “Investigating Mobile Edge-Cloud Trade-Offs of Object Detection with YOLO”. In: *KDMiLe 2019*. 2019. DOI: 10.5753/KDMiLe.2019.8788.
- [Mon+23] Sudip Mondal et al. “Binary Convolutional Neural Network for Efficient Gesture Recognition at Edge”. In: 2023. DOI: 10.1145/3639856.3639874.
- [Pan+25] Ioannis Panopoulos et al. “Greek Sign Language Detection with Artificial Intelligence”. In: *Electronics* 14.16 (2025), p. 3241. DOI: 10.3390/electronics14163241.
- [Pra+24] K. N. Prashanth et al. “Gesture Recognition for Interactive Presentation Control on Raspberry Pi”. In: *Indian Scientific Journal of Research in Engineering and Management* (2024). DOI: 10.55041/ijsrem39636.

- [Raz+23] Mohd Razman et al. “Sign Language Recognition using Deep Learning through LSTM and CNN”. In: *Mekatronika* 5.1 (2023). DOI: 10.15282/mekatronika.v5i1.9410.
- [Rey+] R. Rey et al. “A Performance Analysis of YOLO Models for Deployment on Constrained Edge Devices”. In: *Electronics* 14.3 (), p. 638. DOI: 10.3390/electronics14030638.
- [Sap+24] Rizky Saputra et al. “Recognizing Indonesian Sign Language (Bisindo) Gesture in Complex Backgrounds”. In: *Indonesian Journal of Electrical Engineering and Computer Science* 36.3 (2024), pp. 1583–1593.
- [V+26] R. V et al. “Dynamic Kannada Sign Language Recognition on Resource Constrained Devices”. In: *Scientific Reports* (2026).
- [Zab+] Mahir Zabin et al. “SignNet-Nano: Efficient Sign Language Recognition for Real-Time Edge Deployment”. In: ().
- [Zaf+25] Asim Zafar et al. “Federated Learning-Enhanced Edge Deep Learning Model for EMG-Based Gesture Recognition”. In: *IEEE Sensors Journal* (2025). DOI: 10.1109/jsen.2025.3529841.
- [Zha+23] Mingyue Zhang et al. “A Lightweight Network Deployed on ARM Devices for Hand Gesture Recognition”. In: *IEEE Access* 11 (2023). DOI: 10.1109/ACCESS.2023.3273713.

## A. Hyperparameter Grids and Training Configuration

This appendix provides the complete hyperparameter search spaces and final selected values for all three pipeline configurations, enabling full reproduction of the training runs.

### A.1. Baseline: SVM Grid

| Parameter | Search space                           | Selected value |
|-----------|----------------------------------------|----------------|
| Kernel    | {rbf}                                  | rbf            |
| $C$       | {0.1, 1, 10, 100, 1000}                | 10             |
| $\gamma$  | {scale, auto, 0.001, 0.01, 0.1, 1, 10} | scale          |

Table A.1.: SVM hyperparameter grid (5-fold CV on training set). Feature vector: 36-dim (21 wrist-to-landmark distances + 10 normalised fingertip positions + 5 finger bend angles), standardised with `StandardScaler` fitted on training set only.

### A.2. Option A: EfficientNet-B0 Training Configuration

| Parameter      | Phase 1 (head only, 5 ep) | Phase 2 (full, up to 45 ep)               |
|----------------|---------------------------|-------------------------------------------|
| Optimiser      | AdamW                     | AdamW                                     |
| Learning rate  | $1 \times 10^{-3}$        | $1 \times 10^{-4}$ (cosine to $10^{-6}$ ) |
| Weight decay   | $1 \times 10^{-4}$        | $1 \times 10^{-4}$                        |
| Batch size     | 32                        | 32                                        |
| Early stopping | —                         | patience = 5 on val. accuracy             |
| Dropout        | $p = 0.2$                 | $p = 0.2$                                 |
| Loss           | Cross-entropy             | Cross-entropy                             |
| Input size     | $224 \times 224$          | $224 \times 224$                          |

Table A.2.: EfficientNet-B0 two-phase fine-tuning configuration. ImageNet pretrained weights; classification head replaced by  $1280 \rightarrow 26$  linear layer. Seeds: {42, 0, 123}.

### A.3. Option B: YOLOv8n-detect Training Configuration

Option B is trained end-to-end using SGD with an initial learning rate of 0.01 and cosine annealing, starting from COCO-pretrained YOLOv8n weights. Training runs for up to 100 epochs; early stopping halts it when validation mAP50 has not improved for 20 consecutive

| Parameter             | Value                       |
|-----------------------|-----------------------------|
| Base weights          | COCO pretrained (YOLOv8n)   |
| Optimiser             | SGD                         |
| Initial lr ( $lr_0$ ) | 0.01                        |
| LR schedule           | Cosine annealing            |
| Max epochs            | 100                         |
| Early stopping        | patience = 20 on val. mAP50 |
| Mosaic augmentation   | disabled                    |
| Mixup augmentation    | disabled                    |
| Horizontal flip       | disabled (ASL handedness)   |
| Input size            | $640 \times 640$            |
| Seeds                 | {42, 0, 123}                |

Table A.3.: YOLOv8n end-to-end detection training configuration (Ultralytics `train()` API). 26 output classes (A–Z).

epochs. Mosaic, mixup, and horizontal flip augmentations are all disabled to preserve hand shape and handedness, which carry semantic meaning in ASL.

#### A.4. Data Augmentation and Preprocessing Summary

| Transform                             | Training        | Inference |
|---------------------------------------|-----------------|-----------|
| BGR $\rightarrow$ RGB conversion      | ✓               | ✓         |
| Resize (bilinear)                     | ✓               | ✓         |
| [0, 1] scaling                        | ✓               | ✓         |
| ImageNet normalisation (EfficientNet) | ✓               | ✓         |
| Random rotation $\pm 15^\circ$        | ✓               | —         |
| Brightness/contrast jitter $\pm 20\%$ | ✓               | —         |
| Translation $\pm 10\%$                | ✓               | —         |
| Scaling $0.9\text{--}1.1\times$       | ✓               | —         |
| Horizontal flip                       | <b>disabled</b> | —         |
| Mosaic / mixup                        | <b>disabled</b> | —         |

Table A.4.: Data augmentation policy applied during training. Horizontal flip is unconditionally disabled because handedness carries semantic meaning in ASL. Mosaic and mixup are disabled for Option B because they distort hand shape in ways that degrade recognition accuracy on this task.

Training applies geometric and photometric augmentations (random rotation, brightness and contrast jitter, translation, and scaling) to improve generalisation from the controlled dataset to real signing conditions. Inference uses only the deterministic transforms (colour conversion, resizing, and normalisation). Horizontal flip is unconditionally disabled across all pipelines because left-hand and right-hand signs carry distinct semantic meaning in ASL; mosaic and mixup are additionally disabled for Option B as they distort hand shape in ways that reduce recognition accuracy.